Vývojové metodiky

Techniky vývoje softwaru jsou postupy, které určují, jakým způsobem se vyvíjí software.

Filter by title
Techniky (techniques.html)
+ Balíčky
+ C# (net.html)
JavaScript (javascript.html)

Agilní metodika (Scrum)

- Zaměřuje se na spolupráci, zákaznickou spokojenost a schopnost reagovat na změny.
- V praxi tým pracuje v krátkých iteracích zvaných `sprinty`, které trvají obvykle **2-4 týdny**.

> **NOTE**
> Na konci každého sprintu tým prezentuje hotové funkce zákazníkovi a získává zpětnou vazbu.

> **WARNING**
> Není vhodná pro projekty, které vyžadují pevný plán a jasně definované výstupy.

1. Plánování sprintu (Sprint Planning)
   Týmová schůzka na začátku sprintu, která se koná jednou.

   Výběr úkolů (z backlogu) pro daný sprint k provedení.

   Diskutování o tom, jak budou tyto úkoly provedeny.

2. Vývoj
   Každodenní práce na úkolech, které byly vybrány během plánování sprintu.

   Každodenní týmová schůzka zvaná: `Daily Scrum`, kde se diskutuje o pokroku a případných překážkách na úkolech.

   Testování je součástí této sekce vývoje.

   > **NOTE**
   > Jakmile je úkol nebo funkce vyvinuta, je ihned testována, aby se zajistilo, že funguje správně a splňuje požadavky.

   > **TIP**
   > Toto průběžné testování umožňuje rychlé odhalení a opravu chyb.

3. Revize sprintu (Sprint Review)
   Týmová schůzka na konci sprintu, která se koná jednou.

   Prezentuje se práce zákazníkovi a získává se zpětná vazba.

   > **TIP**

Tato schůzka je také příležitostí k diskusi o tom, co se v sprintu povedlo a co ne.

4. Retrospektiva sprintu (Sprint Retrospective)
   Týmová schůzka obvykle hned po revizi sprintu.

   Diskutuje se o tom, co se v průběhu sprintu povedlo, co se nepovedlo a jak se mohou věci zlepšit v dalším sprintu.

Techniky (techniques.html)

+ Balíčky

+ C# (net.html)

JavaScript (javascript.html)

Vodopádová metodika

Každá fáze musí být dokončena, než může začít další.

WARNING

Délka každé fáze závisí na velikosti a složitosti projektu, počtu lidí v týmu a mnoha dalších faktorech.

1. Analýza požadavků
   Na začátku projektu se shromažďují a analyzují požadavky.

   NOTE

   Toto zahrnuje porozumění potřebám zákazníka a definování, co bude systém dělat.

2. Návrh
   Vytváří se podrobný plán toho, jak bude systém fungovat a jak bude vypadat.
3. Implementace

   Návrh se převede na zdrojový kód.

   Programátoři píší kód, který realizuje návrh systému.
4. Testování

   Kontroluje se, zda systém splňuje požadavky definované v první fázi a zda neobsahuje chyby.
5. Nasazení
   Jakmile je systém otestován a schválen, je nasazen do produkčního prostředí.
6. Údržba
   Sleduje se výkon systému, opravují se chyby a přidávají se nové funkce podle potřeb zákazníka.

Kanban

Vizuální systém řízení práce, který se zaměřuje na dokončení úkolů.

NOTE

Zaměřuje se na průběžnou dodávku a minimalizaci času stráveného na úkolech.

WARNING

Není vhodný pro projekty, které vyžadují pevné plány a termíny

1. Definování úkolů
   Na začátku se vytvoří seznam úkolů, které je třeba dokončit.

   > **TIP**
   > Tyto úkoly se zapisují na karty, které se umístí na Kanban tabuli.

2. Vizualizace práce
   Kanban tabule je rozdělena na několik sloupců, které reprezentují různé stavy úkolu.

   > **TIP**
   > Například: `Backlog` , `Todo` , `Done` , atd...

   Karty s úkoly se přesouvají mezi těmito sloupci podle toho, v jakém stavu se nacházejí.

3. Práce na úkolech
   Tým začne pracovat na úkolech, začínajíc u těch, které jsou nejvíce prioritní.

   > **NOTE**
   > Jakmile je úkol dokončen, karta se přesune do dalšího sloupce.
   >
   > Například sloupec: `Done`

4. Omezení práce v průběhu (Work in Progress, WIP)
   Kanban klade důraz na omezení množství práce, která se může dělat současně.

   > **NOTE**
   > Zvyšuje se efektivita a snižuje se doba, kterou úkol stráví v systému.

5. Průběžné zlepšování
   Tým pravidelně hodnotí svůj proces a hledá způsoby, jak ho zlepšit.

   > **TIP**
   > To může zahrnovat změnu počtu úkolů, prioratizaci úkolů atd...

Rychlé prototypování

Proces pro vytvoření funkčního modelu projektu co nejrychleji, aby bylo možné testovat a iterovat nápady.

> **NOTE**

V kontextu Unity, to znamená vytvoření základní verzi hry nebo aplikace, která zahrnuje pouze klíčové mechaniky a funkce.

NOTE

Rychlá iterace

- Rychle testovat a provádět nápady.
- Pokud vytvoříte prototyp, zkuste ho co nejdříve otestovat a získat zpětnou vazbu.
- Poté můžete na základě této zpětné vazby upravit a vylepšit svůj prototyp.

IMPORTANT

Cílem je vytvořit funkční model vašeho projektu, **ne dokonalý produkt**.

Nebojte se udělat kompromisy v kvalitě, pokud to znamená, že můžete rychleji testovat a iterovat své nápady.

Postup

1. Definice svého konceptu
   Než se začne s prototypováním, měli byste mít jasnou představu o tom, co chcete vytvořit.

   TIP

   To může zahrnovat definování klíčových mechanik, funkcí a cílů vašeho projektu.

2. Vytvořit základní scénu v Unity

   TIP

   Tato scéna bude sloužit jako základ pro váš prototyp.

3. Přidat základní objekty
   Přidejte do scény základní objekty, jako jsou krychle, koule a válce, které můžete použít k reprezentaci různých prvků ve vaší hře.

4. Přidejte mechaniky a funkce
   Použijte skriptování a vestavěné nástroje Unity k přidání mechanik a funkcí do vašeho prototypu.

5. Testujte a iterujte
   Jakmile máte základní prototyp, začněte ho testovat.

   NOTE

   Získejte zpětnou vazbu od ostatních a na základě této zpětné vazby upravte a vylepšujte svůj prototyp.

6. Opakujte proces
   Po provedení změn na svém prototypu ho znovu otestujte a pokračujte v tomto cyklu, dokud nejste spokojeni s výsledkem.

> **TIP**
> Rychlé prototypování je iterativní proces.

Techniky (techniques.html)

Pojmenování BEM

+ Balíčky

BEM = "Block Element Modifier"

+ C# (net.html)

Metodika pro pojmenování tříd v `HTML` a `CSS`.

JavaScript (javascript.html)

> **NOTE**
> Pomáhá udržet váš kód organizovaný a snadno pochopitelný, a to i pro ostatní vývojáře, kteří se na váš kód podívají.

Příklad:

```
<div class="block"> <!-- Block -->
  <div class="block__element"> <!-- Element -->
  </div>
  <div class="block__element--modifier"> <!-- Element with modifier -->
  </div>
</div>
```

```
.block { ... }
.block__element { ... }
.block__element--modifier { ... }
```

Block

Jedná se o samostatnou entitu, která má smysl sama o sobě.

Například: `header`, `container`, `menu`, `checkbox`, atd.

Element

Část bloku, která nemá samostatný význam.

Je semanticky vázána na svůj blok.

Například: `menu item`, `list item`, `checkbox caption`, atd.

Modifier

Varianta bloku nebo prvku, která mění vzhled nebo chování.

Například: `disabled`, `highlighted`, `checked`, atd.

Syntax BEM je poté následující:

- Názvy bloků a elementů jsou odděleny dvěma podtržítky: `__`
- Názvy modifikátorů jsou odděleny dvěma pomlčkami: `--`

Techniky (techniques.html)

+ Balíčky

+ C# (net.html)

JavaScript (javascript.html)

Základy pojmenování při vývoji

Jasné a popisné názvy

Názvy proměnných, funkcí, tříd atd. by měly být dostatečně popisné, aby bylo jasné, co dělají nebo co reprezentují.

Například:

Místo: `p`

použijte: `product`

místo: `calc`

použijte: `calculateAverage`.

> **TIP**
> Vyhněte se používání zkratek a akronymů.

Boolovské proměnné

Boolovské proměnné by měly začínat předponou jako: `is`, `has`, `can` atd., které naznačují, že hodnota může být pravdivá nebo nepravdivá.

Například:

`isAvailable`, `hasPermission`, `canExecute`.

- Názvy funkcí

  Funkce by měly začínat slovesem, které popisuje, co funkce dělá.

  Například:

  `getUserName()`, `calculateTotalPrice()`, `printReport()`.

- Konzistentní názvosloví

  Pokud se rozhodnete pro určitý styl názvosloví, buďte konzistentní v jeho používání po celém kódu.

  Například:

Pokud používáte `camelCase` pro názvy proměnných, používejte je všude.

Jednotné názvy pro stejné typy proměnných

Pokud máte více polí.

Například:

`firstArray` , `secondArray` .

Množné čísla (plurál) pro kolekce

Pokud proměnná reprezentuje kolekci objektů, použijte množné číslo.

Například:

`users` , `products` , `items` .

Konstanty pro "magické" hodnoty

Pokud kód obsahuje "magické" hodnoty, které nejsou okamžitě zřejmé, zvažte jejich nahrazení pojmenovanými konstantami.

Například:

místo: `if (status == 1)`

použijte: `if (status == STATUS_ACTIVE)` .

▼

- **Techniky (techniques.html)**
- **Barvy**
- **C# (net.html)**
- **JavaScript (javascript.html)**

Nuget Packages

- **packages.config**

  Ukládá seznam všech balíčků používaných projektem, **včetně jejich závislostí**.

  Každý balíček je nainstalován do specifické složky projektu (typicky `packages` složka uvnitř řešení).

  > **NOTE**
  > Zde jsou balíčky uloženy v projektu a zkopírovány z globální složky, což může zpomalovat buildy a zvětšuje velikost repozitářů.
  > Používané před rokem 2017. V každém projektu je samostatná složka s balíčky a soubor `.csproj` obsahuje pouze cesty k těmto balíčkům.

- **PackageReference**

  Spravuje pouze balíčky, které jste **přímo nainstalovali**.

  Závislosti se spravují automaticky a nejsou explicitně uvedeny v souboru `.csproj`.

  Balíčky se nestahují do složky projektu, ale do **globální složky balíčků**.

  > **NOTE**
  > Používá balíčky přímo z globální složky `global-packages`, což zrychluje build a snižuje nároky na prostor v projektu.
  > Od roku 2017 je výchozí formát ve Visual Studiu. Všechny balíčky jsou spravovány na jednom centrálním místě a projekt využívá jejich globální umístění, což vede k lepší přehlednosti a výkonu.

Globální složka balíčků

- Balíčky jsou při použití **PackageReference** vždy načítány přímo ze složky **global-packages** (není nutné je kopírovat do projektu, jak to je u packages.config).
- Umístění složky:

  Windows: `%userprofile%\\.nuget\packages`

  Mac/Linux: `~/.nuget/packages`

  > **NOTE**
  > Toto výchozí umístění lze upravit přes proměnné prostředí `NUGET_PACKAGES`

Filter by title

Techniky (techniques.html)

- Balíčky

Nuget Package (nugetPackage.html)

.NET CLI (netCLI.html)

+ C# (net.html)

JavaScript (javascript.html)

.NET CLI (Command Line Interface)

> [!IMPORTAND] Je zapotřebí mít nainstalovaný `.NET SDK (Software Development Kit)` a `.NET Runtime (Framework)`

Filter by title

Pomocná videa k kurzu:

- **Balíčky**

  NugetPackage (nugetPackage.html)
  .NET CLI (netCLI.html)

+ **C# (net.html)**

  JavaScript (javascript.html)

Technicky (technicky.html)

- Windows

  `C:\Program Files\dotnet\`

  TIP
  Použijte tento příkaz:

  ```
  where dotnet
  ```

- macOS

  `/usr/local/share/dotnet/`

  TIP
  Použijte tento příkaz:

  ```
  which dotnet
  ```

- Linux

  `/usr/share/dotnet/` nebo `/usr/local/share/dotnet/`

Záloha globálních nástrojů

1. Zjištění nainstalovaných globálních nástrojů

   ```
   dotnet tool list -g
   ```

   IMPORTANT
   Zaznamenejte názvy a verze nainstalovaných nástrojů, což usnadní jejich opětovnou instalaci.

2. Umístění nástrojů

Zálohujte celý adresář `tools` .

- Windows

  ```
  C:\Users\<uživatelské\_jméno>\.dotnet\tools
  ```

- macOS/Linux

  ```
  ~/.dotnet/tools
  ```

Obnova globálních nástrojů

1. Obnovení ze zálohy

   Pokud máte zálohovaný adresář `tools` , jednoduše zkopírujte jeho obsah zpět do původního umístění.

   > **NOTE**
   > Restartovat příkazový řádek nebo terminál, aby se projevily změny.

2. Kontrola instalace

   Po obnovení zkontrolujte, zda jsou nástroje správně nainstalovány:

   ```
   dotnet tool list -g
   ```

Modifikátory přístupu

Určuje přístup k danému prvku.

Caller's location	public	protected internal	protected	internal	private protected	private	file
Within the file	✔	✔	✔	✔	✔	✔	✔
Within the class	✔	✔	✔	✔	✔	✔	❌
Derived class (same assembly)	✔	✔	✔	✔	✔	❌	❌
Non-derived class (same assembly)	✔	✔	❌	✔	❌	❌	❌
Derived class (different assembly)	✔	✔	✔	❌	❌	❌	❌
Non-derived class (different assembly)	✔	❌	❌	❌	❌	❌	❌

Více podrobností zde (https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers#summary-table).

Složka 'runtimes' a multiplatformní nasazení

Slouží k ukládání **platformově specifických knihoven a binárních souborů**, které jsou nezbytné pro správné spuštění aplikace na různých operačních systémech a architekturách.

Používá se v aplikacích:

- WPF aplikace
- Konzolové aplikace
- ASP.NET Core aplikace
- WinForms aplikace
- .NET knihovny a služby, které se nasazují na různé platformy (Windows, Linux, macOS, atd.)

> **TIP**
>
> V unity se používá složka `'Plugins'` k umístění knihoven (DLL), které jsou platformově specifické
>
> Tato složka může obsahovat nativní kód pro různé platformy (Windows, Android, iOS, macOS, atd.)

Tato složka `'runtime'` se automaticky generuje při publikaci aplikace a hraje zásadní roli zejména při použití dvou typů nasazení.

Každý z těchto typů nasazení určuje, jakým způsobem aplikace zajišťuje dostupnost .NET runtime a dalších závislostí:

self-contained deployment

Aplikace je distribuována společně s **kompletním .NET runtime**

To znamená, že aplikace si nese vlastní runtime a může běžet nezávisle na tom, zda má uživatel na svém systému nainstalovaný správný .NET runtime.

> **TIP**
>
> Toto nasazení je vhodné, pokud chcete zajistit, že aplikace poběží na jakémkoliv počítači, bez ohledu na její aktuální stav.

Výsledkem je větší velikost aplikace, protože obsahuje kompletní runtime pro cílové platformy, které jsou zahrnuty ve složce `'runtimes'`.

V praxi složka `'runtimes'` obsahuje potřebné knihovny a binární soubory pro různé platformy, jako jsou Windows, Linux, macOS, nebo různé architektury (x64, x86, ARM).

Díky tomu může aplikace běžet **out-of-the-box** bez nutnosti další instalace runtime na cílovém systému.

Použití

Nastavit v souboru projektu (.csproj):

```
<PropertyGroup>
    <SelfContained>true</SelfContained>
    <RuntimeIdentifier>win-x64</RuntimeIdentifier> <!-- Nebo jiný RID podle platformy -->
  </PropertyGroup>
```

nebo s více Runtime Identifiers (RID)

```
<PropertyGroup>
    <SelfContained>true</SelfContained>
    <RuntimeIdentifiers>win-x64;linux-x64;osx-x64</RuntimeIdentifiers>
</PropertyGroup>
```

> **NOTE**
>
> **Runtime Identifiers (RID)** Runtime Identifiers (RID) jsou klíčovou součástí procesu nasazení, protože určují, pro jaké platformy a architektury bude aplikace kompilována.
>
> Můžete specifikovat různé RID podle cílové platformy:
>
> - win-x64 (Windows 64-bit)
> - linux-x64 (Linux 64-bit)
> - osx-x64 (macOS 64-bit)
> - win-arm (Windows na ARM procesorech)

Techniky (techniques.html)

+ Balíčky

- C# (net.html)

- Interface (net/net_interface.html)
- Datové typy (net/net_dataTypes.html)
- Atributy (net/net_attributes.html)
- Metody (net/net_methods.html)
- Testy Nunit (net/net_nunit.html)
- Entity Framework (net/net_entityFramework.html)
- WPF (net/net_wpf.html)
- **+ Soubory (net/net_files.html)**

JavaScript (javascript.html)

framework-dependent deployment (runtime-specific deployment)

Aplikace **závisí na přítomnosti .NET runtime** na cílovém systému.

Aplikace neobsahuje runtime v sobě, což zmenšuje její velikost, ale předpokládá, že uživatel má již správnou verzi .NET runtime nainstalovanou.

> **IMPORTANT**
> Pokud runtime není přítomen, aplikace nefunguje, dokud uživatel runtime nedoinstaluje.

V tomto případě složka `"runtimes"` může obsahovat pouze platformově specifické knihovny a závislosti, které nejsou součástí základního .NET runtime, a zajistit kompatibilitu aplikace s různými platformami.

Použití

> **NOTE**
> Nastavte `SelfContained` na `false`, nebo tuto vlastnost úplně vynechte (výchozí nastavení je totiž `framework-dependent`).

Nastavit v souboru projektu (.csproj):

```
<PropertyGroup>
    <SelfContained>false</SelfContained>
</PropertyGroup>
```

Uvolnění zdrojů

- **Řízené zdroje**

  = Objekty, které jsou spravovány garbage collectorem.

  Zahrnuje všechny objekty vytvořené pomocí klíčového slova `new`.

  Garbage collector automaticky sleduje tyto objekty a uvolňuje jejich paměť, když již nejsou potřebné.

  > **TIP**
  > Programátoři nemusí explicitně uvolňovat paměť pro tyto objekty.

- **Neřízené zdroje**

  = Objekty, které nejsou spravovány garbage collectorem.

  Zahrnuje soubory, databázové připojení, síťové zdroje, atd...

  > **NOTE**

Programátoři musí explicitně uvolnit tyto zdroje, aby zabránili úniku paměti.

Uvolnění zdrojů je důležité pro správné fungování aplikace.

Destruktor

- Automaticky volán, když je objekt zničen garbage collectorem.
- Definuje se pomocí syntaxe `~ClassName()`.
- Uvolňuje neřízené zdroje, které třída drží.
- Destruktory nejsou deterministické.

> **TIP**
> Znamená, že nevíme přesně, kdy budou volány.

- **Jsou volány, když garbage collector rozhodne**, že je objekt vhodný ke zničení.

> **NOTE**
> Může to být kdykoliv po tom, co objekt přestane být používán.

Dispose

- Součástí rozhraní `IDisposable` a je určen pro explicitní uvolnění zdrojů.
- Metoda `Dispose` **je volána programátorem**, když je známo, že objekt již nebude potřebný.

> **NOTE**
> To umožňuje okamžité uvolnění zdrojů a zajišťuje, že nebudou drženy déle, než je nutné.

- Metoda `Dispose` je určena pro uvolnění jak řízených, tak neřízených zdrojů.

Volání funkcí z externích DLL

> **NOTE**
> **Platform Invocation Services (PInvoke)** se používá pro volání knihoven z nativního kódu.

- Nativní kód = Kód kompilován do strojového kódu pro konkrétní platformu.
- Strojový kód = Kód přímo spustitelný na hardware dané platformy.

Postup:

1. **Importování funkce z DLL**

   Atribut `DllImport` z `System.Runtime.InteropServices` k importování funkce z DLL.

Například:

```
using System.Runtime.InteropServices;

public class MyProgram
{
    [DllImport("User32.dll")]
    public static extern int MessageBox(IntPtr h, string m, string c, int typ
}
```


Techniky (techniques.html)

+ Balíčky

- C# (net.html)

Interface (net/net_interface.html)

Datové typy (net/net_dataTypes.html)

Atributy (net/net_attributes.html)

Metody (net/net_methods.html)

Testy Nunit (net/net_nunit.html)

Entity Framework (net/net_entityFramework.html)

WPF (net/net_wpf.html)

+ Soubory (net/net_files.html)

JavaScript (javascript.html)

> **NOTE**
>
> `MessageBox` je funkce definovaná v knihovně: `User32.dll`.
>
> Tato funkce je nyní dostupná v rámci tohoto .NET kódu.

Příklad knihovny v c++

Knihovna v C++ by mohla vypadat takto:

```
#include <windows.h>
extern "C" __declspec(dllexport) int MessageBox(HWND h, LPCSTR m, LPCSTR c, int
type)
{
    return MessageBoxA(h, m, c, type);
}
```

Metody a argumenty v c++

- **`extern "C"`**

> **WARNING**
>
> Zajistí, že funkce jsou kompatibilní s C jazykem. (To je důležité pro interoperabilitu mezi C++ a jinými jazyky, jako je C#.)

> **TIP**
>
> Když kompilátor narazí na funkci, změní její název na něco, co jednoznačně identifikuje nejen název funkce, ale také typy jejích parametrů.
>
> To znamená, že název funkce, jak je viděn v DLL, nebude stejný jako název funkce v původním kódu.

Když použijete `extern "C"` , říkáte kompilátoru, aby tuto funkci nezměnil a zachoval její název tak, jak je. To umožňuje jiným jazykům, jako je C#, najít a volat tuto funkci správným názvem.

- **`__declspec(dllexport)`**

Říka kompilátoru C++, že tato funkce nebo proměnná bude exportována z DLL, takže ji může volat jiný kód, který tuto DLL používá.

> **WARNING**
> Důležité k viditelnosti a dostupnosti pro `PInvoke`

- **`HWND`**

"handle to a window" (rukojeť okna)

- **`LPCTSTR`**

"Long Pointer Constant String"

Item	8-bitů (Ansi)	16-bitů (Wide)	Různé
character	CHAR	WCHAR	TCHAR
string	LPSTR	LPWSTR	LPTSTR
string (const)	LPCSTR	LPCWSTR	LPCTSTR

Odkaz zde (https://stackoverflow.com/questions/321413/lpcstr-lpctstr-and-lptstr)

2. **Volání importované funkce**

```
public class MyProgram
{
    public static void Main()
    {
        MessageBox(IntPtr.Zero, "Hello, World!", "Test MessageBox", 0);
    }
}
```

> `IntPtr.Zero`
>
> Konstanta, která reprezentuje nulový ukazatel.
>
> Je to ekvivalent NULL v C++.

`__Internal`

Pokud se používá **`__Internal` jako název DLL v atributu `DllImport`**, znamená to, že funkce se hledá přímo v hlavním spustitelném souboru.

TIP

Hledá tedy v samotné aplikaci, pokud je to nativní kód, nebo v jedné z knihoven, na které aplikace odkazuje.

- Příklad použití v C#:

```
public class MyProgram
{
    [DllImport("__Internal")]
    public static extern int MyFunction();

    public static void Main()
    {
        MyFunction();
    }
}
```

- Příklad definice v C++

```
extern "C" __declspec(dllexport) int MyFunction()
{
    // Implementace vaší funkce
    return 0;
}
```

V tomto příkladu `MyFunction` je funkce definovaná v nativním kódu.

Je tedy součástí aplikace nebo jedné z jejích závislostí.

Tipy

- `PInvoke`

  = **Platform Invocation Services**, což je technika v `.NET`, která umožňuje volání funkcí, které jsou implementovány v neřízeném kódu.

  TIP

  To je obvykle používáno pro volání C API funkcí, které jsou definovány v DLL.

  Příklad:

Techniky (techniques.html)
+ Balíčky
- C# (net.html)
Interface (net/net_interface.html)
Datové typy (net/net_dataTypes.html)
Atributy (net/net_attributes.html)
Metody (net/net_methods.html)
Testy Nunit (net/net_nunit.html)
Entity Framework (net/net_entityFramework.html)
WPF (net/net_wpf.html)
+ Soubory (net/net_files.html)
JavaScript (javascript.html)

```
        private static class PInvoke
        {
        #if UNITY_IOS || UNITY_TVOS
            private const string DllName = "__Internal";
        #elif UNITY_STANDALONE_OSX
            private const string DllName = "MacOSAppleAuthManager";
        #endif

            public delegate void NativeMessageHandlerCallbackDelegate(uint re
        questId, string payload);

            [AOT.MonoPInvokeCallback(typeof(NativeMessageHandlerCallbackDeleg
        ate))]
            public static void NativeMessageHandlerCallback(uint requestId, s
        tring payload)
            {
                try
                {
                    CallbackHandler.ScheduleCallback(requestId, payload);
                }
                catch (Exception exception)
                {
                    Console.WriteLine("Received exception while scheduling a
        callback for request ID " + requestId);
                    Console.WriteLine("Detailed payload:\n" + payload);
                    Console.WriteLine("Exception: " + exception);
                }
            }

            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern bool AppleAuth_IsCurrentPlatformSupported();

            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern void AppleAuth_SetupNativeMessageHandlerCall
        back(NativeMessageHandlerCallbackDelegate callback);

            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern void AppleAuth_GetCredentialState(uint reque
        stId, string userId);

            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern void AppleAuth_LoginWithAppleId(uint request
        Id, int loginOptions, string nonceCStr, string stateCStr);

            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern void AppleAuth_QuickLogin(uint requestId, st
        ring nonceCStr, string stateCStr);

            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern void AppleAuth_RegisterCredentialsRevokedCal
        lbackId(uint callbackId);
```

```
            [System.Runtime.InteropServices.DllImport(DllName)]
            public static extern void AppleAuth_LogMessage(string messageCSt
r);
        }
```

Více info zde (https://github.com/lupidan/apple-signin-unity/blob/master/AppleAuth/AppleAuthManager.cs).

Techniky (techniques.html)

+ Balíčky

- C# (net.html)

Interface (net/net_interface.html)
Datové typy (net/net_dataTypes.html)
Atributy (net/net_attributes.html)
Metody (net/net_methods.html)
Testy Nunit (net/net_nunit.html)
Entity Framework (net/net_entityFramework.html)
WPF (net/net_wpf.html)

+ Soubory (net/net_files.html)

JavaScript (javascript.html)

Interface = rozhraní

ICloneable

Vytvoří kopii objektu

- Mělká

```
public object Clone ()
{
    // Pro referenční typy se kopíruje reference (odkaz), nikoli objekt
    return this.MemberwiseClone();
}
```

- Hluboká

Kopírování referencí jako objekt

> **WARNING**
> Může mít vliv na výkon

- Automaticky:

```
public static T DeepClone<T>(T obj)
{
    using (var ms = new MemoryStream())
    {
        var formatter = new BinaryFormatter();
        formatter.Serialize(ms, obj);
        ms.Position = 0;

        return (T) formatter.Deserialize(ms);
    }
}
```

- Ručně:

```
public class Record : ICloneable
 {
     // ... other properties ...

     public List<string> Tnr { get; set; }
     public List<string> Ean { get; set; }
     // ... other properties ...

     public object Clone()
     {
         return new Record
         {
             // ... clone other properties ...

             Tnr = this.Tnr != null ? new List<string>(this.Tnr) : null,
             Ean = this.Ean != null ? new List<string>(this.Ean) : null,
             // ... clone other properties ...
         };
     }
 }
```

NOTE

Stejným způsob pro klonování vnořených objektů

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Queue

= Fronta

Funguje na principu FIFO (First In First Out) = První prvek, který je vložen do fronty, je první, který je vyjmut.

Filter by title

NOTE

Představte si frontu jako řadu lidí čekajících na autobus.

První člověk, který přišel (`enqueue`), je první, kdo nastoupí do autobusu (`dequeue`).

Příklad:

```
Queue<int> queue = new Queue<int>();
queue.Enqueue(1); // Enqueue() vloží prvek do fronty
queue.Enqueue(2);
queue.Enqueue(3);

Console.WriteLine(queue.Dequeue()); // Dequeue() vyjme první prvek z fronty: 1
Console.WriteLine(queue.Dequeue()); // 2
Console.WriteLine(queue.Dequeue()); // 3
```

PriorityQueue

Každý prvek ve frontě má přiřazenou prioritu.

Prvek s nejvyšší prioritou je vždy vyjmut první.

Příklad:

```
PriorityQueue<int> queue = new PriorityQueue<int>();
queue.Enqueue(1, 2); // Enqueue() vloží prvek do fronty s prioritou 2
queue.Enqueue(2, 1); // Prvek s prioritou 1 bude vyjmut první
queue.Enqueue(3, 3);

Console.WriteLine(queue.Dequeue()); // 2
Console.WriteLine(queue.Dequeue()); // 1
Console.WriteLine(queue.Dequeue()); // 3
```

Stack

= Zásobník

Funguje na principu LIFO (Last In First Out) = Poslední prvek, který je vložen do zásobníku, je první, který je vyjmut.

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy NuNit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

> **NOTE**
>
> Představte si zásobník jako hromadu talířů.
>
> Poslední talíř, který je položen na hromadu (`push`), je první, který je vyjmut (`pop`).

Příklad:

```
Stack<int> stack = new Stack<int>();
stack.Push(1); // Push() vloží prvek na zásobník
stack.Push(2);
stack.Push(3);

Console.WriteLine(stack.Pop()); // Pop() vyjme poslední prvek ze zásobníku: 3
Console.WriteLine(stack.Pop()); // 2
Console.WriteLine(stack.Pop()); // 1
```

- List
  - Testy Nunit
  - Seznam prvků, který lze indexovat.

  Příklad:

```
List<int> list = new List<int>();
list.Add(1); // Add() přidá prvek na konec seznamu
list.Add(2);
list.Add(3);

Console.WriteLine(list[1]); // Vrátí prvek na indexu 1: 2
list.RemoveAt(1); // RemoveAt() odebere prvek na indexu 1
Console.WriteLine(list[1]); // 3
```

- LinkedList
  - `LinkedList`

    Seznam prvků, který lze efektivně upravovat.

    Příklad:

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

```
LinkedList<int> linkedList = new LinkedList<int>();
linkedList.AddLast(1); // AddLast() přidá prvek na konec seznamu
linkedList.AddLast(2);
linkedList.AddLast(3);

Console.WriteLine(linkedList.First.Value); // Vrátí první prvek seznamu: 1
linkedList.RemoveFirst(); // RemoveFirst() odebere první prvek ze seznamu
Console.WriteLine(linkedList.First.Value); // 2
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

> **NOTE**
>
> Rozdíl proti `Dictionary` je, že `LinkedList` je efektivnější pro úpravy, ale méně efektivní pro vyhledávání.

SortedLinkedList

- `SortedLinkedList`

  Seznam prvků, který je automaticky seřazený.

  Příklad:

```
SortedLinkedList<int> sortedLinkedList = new SortedLinkedList<int>();
sortedLinkedList.Add(3); // Add() přidá prvek do seznamu
sortedLinkedList.Add(1);
sortedLinkedList.Add(2);

Console.WriteLine(sortedLinkedList.First.Value); // Vrátí první prvek seznamu: 1
sortedLinkedList.RemoveFirst(); // RemoveFirst() odebere první prvek ze seznamu
Console.WriteLine(sortedLinkedList.First.Value); // 2
```

> **NOTE**
>
> Rozdíl proti `LinkedList` je, že `SortedLinkedList` je automaticky seřazený.

Dictionary

- `Dictionary`

  Kolekce klíč-hodnota, která je efektivní pro vyhledávání.

  Příklad:

```
Map<string, int> map = new Map<string, int>();
map.Put("key1", 1); // Put() přidá klíč a hodnotu do mapy
map.Put("key2", 2);
map.Put("key3", 3);

Console.WriteLine(map.Get("key2")); // Get() vrátí hodnotu pod klíčem "key2": 2
map.Remove("key2"); // Remove() odebere klíč a hodnotu z mapy
Console.WriteLine(map.ContainsKey("key2")); // false
```

> **NOTE**
> Je efektivní pro vyhledávání, ale méně efektivní pro úpravy.

SortedDictionary

- `SortedDictionary`
  Kolekce klíč-hodnota, která je automaticky seřazena podle klíčů.

Příklad:

```
SortedDictionary<string, int> sortedDictionary = new SortedDictionary<string, int>();
sortedDictionary.Add("key3", 3); // Add() přidá klíč a hodnotu do tabulky
sortedDictionary.Add("key1", 1);
sortedDictionary.Add("key2", 2);

Console.WriteLine(sortedDictionary.First().Key); // Vrátí klíč prvního prvku: "key1"
sortedDictionary.Remove("key1"); // Remove() odebere klíč a hodnotu z tabulky
Console.WriteLine(sortedDictionary.First().Key); // "key2"
```

> **NOTE**
> Rozdíl proti `Dictionary` je, že `SortedDictionary` je automaticky seřazený podle klíčů.

HashSet

Množina unikátních prvků.

> **WARNING**
> Není zaručeno, že prvky budou v množině ve stejném pořadí, jak byly přidány.

Příklad:

```
HashSet<int> hashSet = new HashSet<int>();
hashSet.Add(1); // Add() přidá prvek do množiny
hashSet.Add(2);
hashSet.Add(3);

Console.WriteLine(hashSet.Contains(2)); // Contains() zjistí, zda množina obsahuje prvek: true
hashSet.Remove(2); // Remove() odebere prvek z množiny
Console.WriteLine(hashSet.Contains(2)); // false
```

> **NOTE**
>
> `HashSet` je optimalizovaný pro rychlé vyhledávání.

Hashtable

Kolekce klíč-hodnota.

Příklad:

```
Hashtable hashtable = new Hashtable();
hashtable.Add("key1", 1); // Add() přidá klíč a hodnotu do tabulky
hashtable.Add("key2", 2);
hashtable.Add("key3", 3);

Console.WriteLine(hashtable["key2"]); // Vrátí hodnotu pod klíčem "key2": 2
hashtable.Remove("key2"); // Remove() odebere klíč a hodnotu z tabulky
Console.WriteLine(hashtable.Contains("key2")); // false
```

> **WARNING**
>
> `Hashtable` je starší třída a je obecně doporučováno používat `Dictionary`

Tuple

Kolekce prvků různých typů.

Příklad:

```
Tuple<int, string> tuple = new Tuple<int, string>(1, "Hello");
Console.WriteLine(tuple.Item1); // Vrátí první prvek: 1
Console.WriteLine(tuple.Item2); // Vrátí druhý prvek: "Hello"
```

> **NOTE**

`Tuple` je obecný typ, který lze použít pro vytvoření kolekce prvků různých typů.

ValueTuple

Kolekce prvků různých typů, která je optimalizovaná pro rychlé vytváření.

Kolekce znamená, že prvek může obsahovat více prvků.

Příklad:

```
(int, string) valueTuple = (1, "Hello");
Console.WriteLine(valueTuple.Item1); // Vrátí první prvek: 1
Console.WriteLine(valueTuple.Item2); // Vrátí druhý prvek: "Hello"
```

NOTE

`ValueTuple` je optimalizovaný typ, který lze použít pro rychlé vytváření kolekce prvků různých typů.

Vlastní názvy prvků:

```
var tuple = (Shield: 1, Sword: "Hello");
Console.WriteLine(tuple.Shield); // Vrátí první prvek: 1
Console.WriteLine(tuple.Sword); // Vrátí druhý prvek: "Hello"
```

ObservableCollection

Kolekce, která upozorní na změnu prvků.

Příklad:

```
ObservableCollection<int> observableCollection = new ObservableCollection<int>();
observableCollection.CollectionChanged += (sender, e) => {
    Console.WriteLine("Collection changed");
};
observableCollection.Add(1); // Přidá prvek do kolekce
```

NOTE

`ObservableCollection` je užitečná pro sledování změn v kolekci.

WeakReference

Slabá reference na objekt, která nezabraňuje garbage collectoru v uvolnění paměti.

Příklad:

```
WeakReference<int> weakReference = new WeakReference<int>(1);
Console.WriteLine(weakReference.TryGetTarget(out int value)); // TryGetTarget()
```

NOTE

`WeakReference` je užitečná pro sledování objektů, které mohou být uvolněny garbage collectorem.

Memory

Ukazatel na paměť, který umožňuje bezpečný přístup k paměti.

Příklad:

```
Memory<int> memory = new Memory<int>(new int[] { 1, 2, 3 });
Console.WriteLine(memory.Span[1]); // Span[] vrátí prvek na indexu 1: 2
```

NOTE

`Memory` je užitečná pro bezpečný přístup k paměti.

Span

Ukazatel na paměť, který umožňuje bezpečný přístup k paměti.

Příklad:

```
Span<int> span = new Span<int>(new int[] { 1, 2, 3 });
Console.WriteLine(span[1]); // Vrátí prvek na indexu 1: 2
```

NOTE

`Span` je užitečná pro bezpečný přístup k paměti.

Immutable kolekce

ImmutableArray

Pole, které nelze měnit.

Příklad:

```
ImmutableArray<int> immutableArray = ImmutableArray.Create(1, 2, 3);
Console.WriteLine(immutableArray[1]); // Vrátí prvek na indexu 1: 2
```

> **NOTE**
> `ImmutableArray` je pole, které nelze měnit.

Immutable List

Seznam, který nelze měnit.

Příklad:

```
ImmutableList<int> immutableList = ImmutableList.Create(1, 2, 3);
Console.WriteLine(immutableList[1]); // Vrátí prvek na indexu 1: 2
```

> **NOTE**
> `ImmutableList` je seznam, který nelze měnit.

Immutable Dictionary

Kolekce klíč-hodnota, která nelze měnit.

Příklad:

```
    ImmutableDictionary<string, int> immutableDictionary = ImmutableDictionary.Creat
e<string, int>();
    immutableDictionary = immutableDictionary.Add("key1", 1); // Add() přidá klíč a
hodnotu do tabulky
    immutableDictionary = immutableDictionary.Add("key2", 2);
    immutableDictionary = immutableDictionary.Add("key3", 3);

    Console.WriteLine(immutableDictionary["key2"]); // Vrátí hodnotu pod klíčem "key
2": 2
    immutableDictionary = immutableDictionary.Remove("key2"); // Remove() odebere kl
íč a hodnotu z tabulky
    Console.WriteLine(immutableDictionary.ContainsKey("key2")); // false
```

> **NOTE**

`ImmutableDictionary` je kolekce klíč-hodnota, která nelze měnit.

Immutable HashSet

Množina prvků, kterou nelze měnit.

Příklad:

```
ImmutableHashSet<int> immutableHashSet = ImmutableHashSet.Create(1, 2, 3);
Console.WriteLine(immutableHashSet.Contains(2)); // Contains() zjistí, zda množina obsahuje prvek: true
```

NOTE

`ImmutableHashSet` je množina prvků, kterou nelze měnit.

Immutable SortedSet

Seřazená množina prvků, kterou nelze měnit.

Příklad:

```
ImmutableSortedSet<int> immutableSortedSet = ImmutableSortedSet.Create(3, 1, 2);
Console.WriteLine(immutableSortedSet.Min); // Min vrátí nejmenší prvek: 1
```

NOTE

`ImmutableSortedSet` je seřazená množina prvků, kterou nelze měnit.

Immutable Stack

Zásobník prvků, který nelze měnit.

Příklad:

```
ImmutableStack<int> immutableStack = ImmutableStack.Create(1, 2, 3);
Console.WriteLine(immutableStack.Peek()); // Peek() vrátí poslední prvek: 3
```

NOTE

`ImmutableStack` je zásobník prvků, který nelze měnit.

Immutable Queue

Fronta prvků, kterou nelze měnit.

Příklad:

```
ImmutableQueue<int> immutableQueue = ImmutableQueue.Create(1, 2, 3);
Console.WriteLine(immutableQueue.Peek()); // Peek() vrátí první prvek: 1
```

NOTE

`ImmutableQueue` je fronta prvků, kterou nelze měnit.

Readonly kolekce

ReadOnlyCollection

Kolekce, kterou nelze měnit.

Příklad:

```
List<int> list = new List<int> { 1, 2, 3 };
ReadOnlyCollection<int> readOnlyCollection = new ReadOnlyCollection<int>(list);
Console.WriteLine(readOnlyCollection[1]); // Vrátí prvek na indexu 1: 2
```

NOTE

`ReadOnlyCollection` je kolekce, kterou nelze měnit.

ReadOnlyDictionary

Kolekce klíč-hodnota, kterou nelze měnit.

Příklad:

```
Dictionary<string, int> dictionary = new Dictionary<string, int> {
    { "key1", 1 },
    { "key2", 2 },
    { "key3", 3 }
};
ReadOnlyDictionary<string, int> readOnlyDictionary = new ReadOnlyDictionary<string, int>(dictionary);
Console.WriteLine(readOnlyDictionary["key2"]); // Vrátí hodnotu pod klíčem "key2": 2
```

> **NOTE**
>
> `ReadOnlyDictionary` je kolekce klíč-hodnota, kterou nelze měnit.

ReadOnlyList

Seznam prvků, který nelze měnit.

Příklad:

```
List<int> list = new List<int> { 1, 2, 3 };
ReadOnlyList<int> readOnlyList = new ReadOnlyList<int>(list);
Console.WriteLine(readOnlyList[1]); // Vrátí prvek na indexu 1: 2
```

> **NOTE**
>
> `ReadOnlyList` je seznam prvků, který nelze měnit.

Asynchronní struktury dat

ConcurrentQueue

Fronta, která je bezpečná pro použití ve více vláknech.

Příklad:

```
ConcurrentQueue<int> concurrentQueue = new ConcurrentQueue<int>();
concurrentQueue.Enqueue(1); // Enqueue() vloží prvek do fronty
concurrentQueue.Enqueue(2);
concurrentQueue.Enqueue(3);

Console.WriteLine(concurrentQueue.TryDequeue(out int value)); // TryDequeue() vy
jme první prvek z fronty: true
Console.WriteLine(value); // 1
```

> **NOTE**
>
> `ConcurrentQueue` je bezpečná pro použití ve více vláknech.

ConcurrentStack

Zásobník, který je bezpečný pro použití ve více vláknech.

Příklad:

```
ConcurrentStack<int> concurrentStack = new ConcurrentStack<int>();
concurrentStack.Push(1); // Push() vloží prvek na zásobník
concurrentStack.Push(2);
concurrentStack.Push(3);

Console.WriteLine(concurrentStack.TryPop(out int value)); // TryPop() vyjme poslední prvek ze zásobníku: true
Console.WriteLine(value); // 3
```


> **NOTE**
>
> `ConcurrentStack` je bezpečný pro použití ve více vláknech.

ConcurrentDictionary

Kolekce klíč-hodnota, která je bezpečná pro použití ve více vláknech.

Příklad:

```
ConcurrentDictionary<string, int> concurrentDictionary = new ConcurrentDictionary<string, int>();
concurrentDictionary.TryAdd("key1", 1); // TryAdd() přidá klíč a hodnotu do tabulky
concurrentDictionary.TryAdd("key2", 2);
concurrentDictionary.TryAdd("key3", 3);

Console.WriteLine(concurrentDictionary["key2"]); // Vrátí hodnotu pod klíčem "key2": 2
concurrentDictionary.TryRemove("key2", out _); // TryRemove() odebere klíč a hodnotu z tabulky
Console.WriteLine(concurrentDictionary.ContainsKey("key2")); // false
```

> **NOTE**
>
> `ConcurrentDictionary` je bezpečná pro použití ve více vláknech.

ConcurrentBag

Kolekce prvků, která je bezpečná pro použití ve více vláknech.

Příklad:

```
ConcurrentBag<int> concurrentBag = new ConcurrentBag<int>();
concurrentBag.Add(1); // Add() přidá prvek do kolekce
concurrentBag.Add(2);
concurrentBag.Add(3);

Console.WriteLine(concurrentBag.TryTake(out int value)); // TryTake() vyjme prvek z kolekce: true
Console.WriteLine(value); // 1
```

NOTE

`ConcurrentBag` je bezpečná pro použití ve více vláknech.

BlockingCollection

Kolekce prvků, která blokuje vlákno, pokud je prázdná nebo plná.

Příklad:

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>();
Task.Run(() => {
    blockingCollection.Add(1); // Add() přidá prvek do kolekce
});
Console.WriteLine(blockingCollection.Take()); // Take() vyjme prvek z kolekce
```

NOTE

`BlockingCollection` je užitečná pro synchronizaci mezi vlákny.

Atributy obsahují **Metadata**

Datové anotace

= System.ComponentModel.Annotations (namespace)

Filter by title

Nejvíce používané anotace:

- `[Required]`

  Hodnota je povinná
- `[Range]`

  Hodnota v číselném rozsahu
- `[MaxLength]`

  Hodnota s maximální délkou
- `[MinLength]`

  Hodnota s minimální délkou
- `[StringLength]`

  Hodnota s maximální a volitelnou minimální délkou
- `[RegularExpression]`

  Ověření hodnoty z regulárního výrazu
- `[DataType]`

  Specifikuje datový typ pro hodnotu
- `[Display]`

  Název a pořadí hodnoty

Příklad

- Definice

```
public class Author
{
    [Required(ErrorMessage = "{0} is required")]
    [StringLength(50, MinimumLength = 3,
    ErrorMessage = "First Name should be minimum 3 characters and a maximum of
50 characters")]
    [DataType(DataType.Text)]
    public string FirstName { get; set; }

    [Required(ErrorMessage = "{0} is required")]
    [StringLength(50, MinimumLength = 3,
    ErrorMessage = "Last Name should be minimum 3 characters and a maximum of 5
0 characters")]
    [DataType(DataType.Text)]
    public string LastName { get; set; }

    [DataType(DataType.PhoneNumber)]
    [Phone]
    public string PhoneNumber { get; set; }

    [DataType(DataType.EmailAddress)]
    [EmailAddress]
    public string Email { get; set; }
}
```

- Použití

```
Author author = new Author();
author.FirstName = "Joydip";
author.LastName = "";
author.PhoneNumber = "1234567890";
author.Email = "joydipkanjilal@yahoo.com";

// Provedení kontroly dat
ValidationContext context = new ValidationContext(author, null, null);
List<ValidationResult> validationResults = new List<ValidationResult>();
bool valid = Validator.TryValidateObject(author, context, validationResults, tr
ue);
if (!valid)
{
  foreach (ValidationResult validationResult in validationResults)
  {
       Console.WriteLine("{0}", validationResult.ErrorMessage);
  }
}
```

Vlastní datová anotace

1. Vytvořit třídu a rozšířit ji o třídu `ValidationAttribute`

2. Přepsat metodu `IsValid`

- Definice

```
[AttributeUsage(AttributeTargets.Property, AllowMultiple = false, Inherited = f
alse)]
public class IsEmptyAttribute : ValidationAttribute
{
    public override bool IsValid(object value)
    {
        var inputValue = value as string;
        return !string.IsNullOrEmpty(inputValue);
    }
}
```

- Použití

```
[IsEmpty(ErrorMessage = "Should not be null or empty.")]
public string FirstName { get; set; }

[IsEmpty(ErrorMessage = "Should not be null or empty.")]
public string LastName { get; set; }
```

Použité odkazy [1]

FileHelpers

= Filehelpers namespace)

IMPORTANT

Nepodporuje:

- Záznamy s proměnnou délkou

  = každý záznam musí mít stejný počet polí

- Změnu formátu za běhu

  = každý záznam musí mít stejný formát po celou dobu.

Nejvíce používané atributy:

- Pro třídu:
  - `[DelimitedRecord]`

    Záznamy s oddělovači.

    Argument: řetězec reprezentující oddělovač pro záznamy.

  - `[FixedLengthRecord]`

Záznamy s pevnou délkou.

- Pro pole
  - `[FieldTrim]`

    Odstranění bílých znaků z hodnoty.

    Argument: `TrimMode.Both` , `TrimMode.Left` , `TrimMode.Right` .

    - `[FieldOptional]`

      Volitelný sloupec.

      Pokud sloupec v souboru chybí, nebude to považováno za chybu.

    - `[FieldIgnore]`

      Ignoruje sloupec při čtení nebo zápisu souboru.

    - `[FieldConverter]`

      Přiřazení konvertoru.

      Konvertor = třída, převádí hodnoty mezi textovou reprezentací v souboru a hodnotou v datech.

    - `[FieldOrder]`

      Určuje pořadí sloupců pro čtení nebo zápis souboru.

    - `[FieldQuoted]`

      Pokud je hodnota v souboru uvedena v uvozovkách.

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Příklad

1. Definice třídy pro záznamy
   - CSV soubory

     [DelimitedRecord(",")]

   - Soubory s pevnou délkou záznamu

     [FixedLengthRecord]

2. Přečíst nebo zapsat soubor - Čtení ze souboru

   ```c#
   var engine = new FileHelperEngine<Order>();
   Order[] result = engine.ReadFile("Input.txt");
   ```

   - Zápis do souboru

     ```
     var engine = new FileHelperEngine<Order>();
     engine.WriteFile("Output.txt", result);
     ```

Vlastní konvertor

1. Vytvořit třídu a rozšířit ji o třídu `ConverterBase`
2. Přepsat metody `StringToField` a `FieldToString.`

- Definice

```
public class MyCustomConverter : ConverterBase
{
    public override object StringToField(string from)
    {
        // Převeďte řetězec na objekt
    }

    public override string FieldToString(object fieldValue)
    {
        // Převeďte objekt na řetězec
    }
}
```

- Použití

```
[DelimitedRecord(",")]
public class Order
{
    [FieldConverter(typeof(MyCustomConverter))]
    public int OrderID;

    // ...
}
```

[^1]: How to use data annotations in C# | InfoWorld (https://www.infoworld.com/article/3543302/how-to-use-data-annotations-in-c-sharp.html)

Základní pojmy

- Parametr

  proměnná v definici metody

Filter by title

- Argument

  skutečná hodnota této proměnné, která je předána při volání metody.

Techniky (../techniques.html)

+ Balíčky

Předání reference či hodnoty do metody

- C# (../net.html)

- `Value`

  Interface (../net/net_interface.html)

  Datové typy (../net/net_dataTypes.html)

  Atributy (../net/net_attributes.html)

  Metody (../net/net_methods.html)

  Testy Nunit (../net/net_nunit.html)

  Entity Framework (../net/net_entityFramework.html)

  WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Pokud předáváte data hodnotou, **vytvoří se kopie dat** a ta se předá do metody.

> **WARNING**
>
> Jakákoli změna dat v metodě **neovlivní původní data**.
>
> Pro velké struktury nebo třídy může být kopírování dat náročné na paměť a čas.

> **NOTE**
>
> **Efektivní pro malé datové typy**
>
> Jako jsou `int`, `float`, `bool` atd...
>
> Kopírování těchto malých hodnot je rychlé.

- `Reference`

Pokud předáte data odkazem pomocí klíčových slov `ref`, `out` nebo `in`, místo kopírování dat se **předá odkaz na původní data**.

> **WARNING**
>
> Metoda může přímo **manipulovat s původními daty**.
>
> **Neplatí pouze pro klíčové slovo `in`**, jelikož ta umožňuje pouze čtení dat.

> **NOTE**
>
> **Efektivní pro velké struktury nebo třídy**, protože se vyhnete nákladnému kopírování dat.
>
> Předá se jen malý odkaz na data.

Druhy metod

Indexátor

Speciální druh členské funkce v C#, která umožňuje objektům být indexovány jako pole.

Příklad použití:

- Definice

```
/// <summary> OAuth2 library configuration. </summary>
public interface IOAuth2Configuration
{
    /// <summary> Returns settings for service client with given name. </summar
    y>
    IClientConfiguration this[string clientTypeName] { get; }
}
```

- Volání

```
IOAuth2Configuration config = ...
IClientConfiguration clientConfig = config["clientTypeName"];
```

> **NOTE**
> Není zapotřebí vytvářet tedy novou funkci jako například: `GetClientConfiguration(string clientTypeName)`.

Statické metody

Deklarovány s klíčovým slovem **`static`**.

> **NOTE**
> Mohou být volány bez vytváření instance třídy.

```
public static void MyStaticMethod()
{
    // kód metody
}
```

Instanční metody

> **WARNING**

Vyžaduje vytvořenou instanci třídy pro jejich volání.

Instance třídy = Objekt vytvořený z dané třídy.

```
public void MyInstanceMethod()
{
    // kód metody
}
```


Virtuální metody

Deklarovány s klíčovým slovem **`virtual`**

Umožňují potomkům třídy přepsat jejich implementaci.

```
public virtual void MyVirtualMethod()
{
    // kód metody
}
```

Abstraktní metody

Deklarovány s klíčovým slovem **`abstract`**

WARNING

Nemají žádnou vlastní implementaci.

Implementaci musí poskytnout třída, která dědí z abstraktní třídy.

```
public abstract void MyAbstractMethod();
```

Přetížené metody

Metody se stejným názvem, které se liší v počtu nebo typu parametrů.

```
public void MyMethod(int param1)
{
    // kód metody
}

public void MyMethod(int param1, int param2)
{
    // kód metody
}
```

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Metody s výchozími hodnotami

Jedná se o parametry, které mohou být při volání metody vynechány.

> **WARNING**
> Parametry s výchozími hodnotami lze použít až po všech nevýchozích parametrech.
> Neplatí pro parametr s klíčovým slovem `params`, ten musí být vždy poslední v seznamu parametrů.

```
public void MyMethod(int param1, int param2 = 10)
{
    // kód metody
}
```

> Při volání této metody by mohl být vynechán `param2`.
>
> Pokud by tento parametr byl vynechán, použila by se jeho výchozí hodnota tedy = 10.

Metody s parametry params

Mohou přijímat **libovolný počet parametrů stejného typu**.

> **WARNING**
> Musí být **vždy poslední v seznamu parametrů**.

```
public void MyMethod(params int[] numbers)
{
    // kód metody
}
```

Metody s výstupními (out) a referenčními (ref) parametry

Mohou měnit hodnotu argumentů předaných do metody.

```
public void MyMethod(out int param1, ref int param2)
{
    // kód metody
}
```

ref

Klíčové slovo `ref` se používá k označení parametru, který se má předat odkazem.

> **NOTE**
> Pokud metoda změní hodnotu ref parametru, **změna se projeví i na původní proměnné**, která byla předána do metody.

> **WARNING**
> Musí být inicializovány před jejich předáním do metody.

Příklad:

```
public void AddTen(ref int number)
{
    number += 10;
}

int myNumber = 5;
AddTen(ref myNumber);  // myNumber je nyní 15
```

out

Klíčové slovo `out` se používá k označení parametru, který se má předat odkazem a bude použit k **vrácení dat z metody**.

> **WARNING**
> Musí být nastaveny v rámci metody před jejím ukončením.

> Užitečné v situacích, kdy chcete, aby metoda vrátila více než jednu hodnotu.
>
> Nemusí být inicializovány před jejich předáním do metody.

Příklad:

```
public void GetValues(out int x, out int y)
{
    x = 5;
    y = 10;
}

int a, b;
GetValues(out a, out b);  // a je nyní 5, b je nyní 10
```

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

in

Klíčové slovo `in` se používá k označení parametru, který se má předat odkazem, ale **nemůže být změněn metodou**.

> Užitečné pro předání velkých struktur nebo tříd, protože se předávají odkazem (efektivněji) a bez rizika, že by metoda změnila jejich hodnotu.

Příklad:

```
public void PrintValue(in int number)
{
    Console.WriteLine(number);
    // number = 10;  // Toto by způsobilo chybu kompilace
}

int myNumber = 5;
PrintValue(in myNumber);  // Vypíše 5
```

Rozšířené metody

Rozšiřují existující typy o nové metody bez nutnosti dědění nebo změny původního typu.

```
public static class MyExtensionMethods
{
    public static void MyExtensionMethod(this MyType myType)
    {
        // kód metody
    }
}
```

Asynchronní metody

Deklarovány s klíčovým slovem **`async`** .

Umožňují asynchronní operace.

> Asynchronní operace
>
> Běží nezávisle na hlavním programu.
>
> Umožňuje tedy hlavnímu programu pokračovat v práci, aniž by musel čekat na dokončení operace.

```
public async Task MyAsyncMethod()
{
    // Kod metody
}
```

> Je sepsána samostatná sekce, kde se lze dozvědět více.

Ukazatel na metody

Delegáti

Drží **odkazy na metody**.

Když je delegát volán, volá se metoda, na kterou odkazuje.

> Typově bezpečný.
>
> Bezpečně tedy zapouzdřují metodu nebo sadu metod.

> Užitečné pro implementaci událostí a zpětných volání.
>
> Jsou základem pro LINQ dotazy a asynchronní metody.

Příklad:

```
// Definice delegáta
public delegate void MyDelegate(string message);

// Metoda, která odpovídá signatuře delegáta
public void MyMethod(string message)
{
    Console.WriteLine(message);
}

// Použití delegáta
MyDelegate del = MyMethod;
del("Hello, World!");  // Vypíše "Hello, World!" na konzoli
```

Delegáti Generické

> **WARNING**
> Je zapotřebí znalost základních delegátů

Func

Delegát pro metody, který **vrací hodnotu**.

- `Func<TResult>`

  Představuje metodu, která vrací hodnotu typu `TResult`.

- `Func<T, TResult>`

  Představuje metodu, která přijímá jeden argument typu `T` a vrací hodnotu typu `TResult`.

> `Func<T1,T2,T3...TResult>` umožňuje až 16 vstupních parametrů.

Příklad:

```
// Using Func delegate
Func<Player, int, int> Attack = (p, damage) =>
{
    p.Health -= damage;
    Console.WriteLine($"{p.Name} is attacked and lost {damage} health.");
    return p.Health;
};

int remainingHealth = Attack(player, 20);
```

Action

Delegát, který **nevrací hodnotu**.

- `Action`

  Představuje metodu bez parametrů.

- `Action<T>`

  Představuje metodu, která přijímá jeden argument typu `T`.

> `Action<T1, T2, T3...>` umožňuje až 16 vstupních parametrů.

Příklad:

```
Action<string> actionMethod = message => Console.WriteLine(message);

// Použití delegáta k zobrazení zprávy
actionMethod("Hello, World!");
```

Predicate

Reprezentuje metodu, která **přijímá jeden argument typu `T`** a **vrací `bool`**.

> Je to speciální případ `Func<T, bool>`.

Příklad:

```
Predicate<int> isEven = x => x % 2 == 0;
bool result = isEven(4);  // result je true
```

Asynchronní a Paralelní metody

Umožňuje vykonávání operací bez blokování (nebo "zamrzání") hlavního vlákna aplikace.

> **NOTE**
>
> Kód může pokračovat v dalších úlohách, zatímco asynchronní operace běží "na pozadí".
>
> Mohou být přerušeny, aby uvolnily vlákno pro jiné úkoly.

> **TIP**
>
> Klíčové slova `async` pro označení metod jako asynchronních a `await` pro čekání na dokončení asynchronních operací nebo úloh.

Příklady použití

- I/O operace:

  Čtení/zápis souborů, síťové požadavky, přístup k databázi.

- Časově náročné výpočty:

  Operace, které trvají dlouho a mohou být prováděny na pozadí.

- Čekání na události:

  Čekání na uživatelský vstup nebo jiné události.

Vytvoření metody

- Bez návratové hodnoty

```
public async Task MyAsyncMethod()
{
    // Zde můžete provést nějakou synchronní práci
    int x = 10;
    int y = 20;
    int sum = x + y;

    // Použijte Task.CompletedTask k naplnění smlouvy asynchronní metody
    await Task.CompletedTask;
}
```

- S návratovou hodnotou

```
public async Task<int> MyAsyncMethod()
{
    // Zde můžete provést nějakou synchronní práci
    int x = 10;
    int y = 20;
    int sum = x + y;
       // Použijte Task.FromResult k naplnění smlouvy asynchronní metody
    return await Task.FromResult(sum);
}
```


Techniky (../techniques.html)

+ Balíčky

C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Volání v asynchronní metodě

```
public async Task MethodA()
{
    // Nějaký kód...

    // Volání MethodB z MethodA
    await MethodB();

    // Další kód...
}

public async Task MethodB()
{
    // Nějaký kód...

    // Simulace asynchronní operace
    await Task.Delay(1000);

    // Další kód...
}
```

Volání v synchronní metodě

> **WARNING**
> Nedoporučuje se používat asynchronní metody v synchronní metodě pokud je to možné.

- Zpracovat výsledek v novém vlákně

```
public void MySyncMethod()
{
    // Zde můžete provést nějakou synchronní práci
    int x = 10;
    int y = 20;
    int sum = x + y;

    // Volání asynchronní metody ve vlákně ThreadPool
    Task task = Task.Run(async () => await MyAsyncMethod());

    // Počkejte na dokončení úkolu
    task.Wait();
}
```

```
private void OnEnable()
{
    // Spuštění asynchronní operace
    Task<T> task = MyAsyncMethod();

    // Zpracování výsledku po dokončení úlohy (také se spustí asynchronně)
    task.ContinueWith(t =>
    {
        if (t.IsFaulted)
        {
            // Zpracování všech výjimek
            Debug.LogError(t.Exception);
        }
        else
        {
            // Použijte výsledek
            result = t.Result;
            // ...
        }
    });
}
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

TIP

Metoda `ContinueWith` je použita pro plánování další operace, která se má vykonat po dokončení úlohy.

Pokud metoda narazí na `ContinueWith`, vytvoří se nový úkol, který se spustí po dokončení původní úlohy.

Obvykle se spustí na jiném vlákně, než je hlavní vlákno. (To je dáno tím, jak .NET spravuje svůj thread pool.)

- Zpracovat výsledek v hlavním vlákně

WARNING

Nedoporučuje se.

Může to vést k problémům s výkonem a odezvou aplikace.

TIP

Hlavní vlákno je obvykle vlákno, které zpracovává události uživatelského rozhraní (UI) a další kritické operace.

Pokud hlavní vlákno zablokujete čekáním na dokončení asynchronní operace, může to způsobit, že se vaše aplikace "zasekne" nebo přestane reagovat na vstupy uživatele.

```
private void OnEnable()
{
    // Spuštění asynchronní operace
    Task<T> task = MyAsyncMethod();

    // Tím se zablokuje aktuální vlákno, dokud nebude asynchronní operace dokon
čena.
    T result = task.GetAwaiter().GetResult();
}
```


Task Parallel Library (TPL)

TPL je sada API, které umožňují paralelní programování.

TIP

TPL zahrnuje `Task` a `Task<T>` třídy, které reprezentují jednotlivé operace, které mohou být asynchronní a mohou vrátit hodnotu.

TPL také zahrnuje `Parallel` třídu pro paralelní iterace a regiony.

Typy metod k použití

asynchronní a paralelní

- Vytváření a spouštění úloh

```
Task task = Task.Run(async () =>
{
    // Kód úlohy
    await SomeAsyncMethod();
});
```

- Čekání na dokončení úlohy

```
Task task = Task.Run(async () =>
{
    // Kód úlohy
    await SomeAsyncMethod();
});
await task;
```

- Získání výsledku úlohy

```
Task<int> task = Task.Run(async () =>
{
    // Kód úlohy
    return await SomeAsyncMethodReturningInt();
});
int result = await task;
```

- Řetězení úloh

```
Task<int> task = Task.Run(async () =>
{
    // Kód první úlohy
    return await SomeAsyncMethodReturningInt();
}).ContinueWith(async previousTask =>
{
    // Kód druhé úlohy
    return await SomeOtherAsyncMethod(previousTask.Result);
});
int result = await task;
```

- Paralelní cykly

```
var items = Enumerable.Range(0, 10);
var tasks = items.Select(async i =>
{
    // Kód pro každou iteraci
    await SomeAsyncMethod(i);
});
await Task.WhenAll(tasks);
```

- Paralelní zpracování kolekcí

```
var items = new List<int> { 1, 2, 3, 4, 5 };
var tasks = items.Select(async item =>
{
    // Kód pro každou položku
    await SomeAsyncMethod(item);
});
await Task.WhenAll(tasks);
```

- Paralelní spuštění více operací

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

```
var tasks = new[]
{
    Task.Run(async () => { await SomeAsyncMethod(); }),
    Task.Run(async () => { await SomeOtherAsyncMethod(); }),
    Task.Run(async () => { await YetAnotherAsyncMethod(); })
};
await Task.WhenAll(tasks);
```

Techniky (../techniques.html)

- Asynchronní metody s `async` a `await`

```
public async Task DoWorkAsync()
{
    await Task.Run(async () =>
    {
        // Kód úlohy
        await SomeAsyncMethod();
    });
}
```

Synchronní a paralelní

- Vytváření a spouštění úloh

```
Task task = Task.Run(() =>
{
    // Kód úlohy
});
```

- Čekání na dokončení úlohy

```
Task task = Task.Run(() =>
{
    // Kód úlohy
});
task.Wait();
```

- Získání výsledku úlohy

```
Task<int> task = Task.Run(() =>
{
    // Kód úlohy
    return 42;
});
int result = task.Result;
```


+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

- Řetězení úloh

```
Task<int> task = Task.Run(() =>
{
    // Kód první úlohy
    return 42;
}).ContinueWith(previousTask =>
{
    // Kód druhé úlohy
    return previousTask.Result * 2;
});
int result = task.Result;
```

- Paralelní cykly

```
Parallel.For(0, 10, i =>
{
    // Kód pro každou iteraci
});
```

- Paralelní zpracování kolekcí

```
var items = new List<int> { 1, 2, 3, 4, 5 };
Parallel.ForEach(items, item =>
{
    // Kód pro každou položku
});
```

- Paralelní spuštění více operací

```
Parallel.Invoke(
    () => { /* Kód první operace */ },
    () => { /* Kód druhé operace */ },
    () => { /* Kód třetí operace */ }
);
```

Příklady

- Spustit v asynchronním módu

```
using System.Threading.Tasks;

public class TPLExample
{
    public async Task RunTasksAsync()
    {
        // Vytvoření a spuštění úloh paralelně
        Task task1 = DoWorkAsync(1);
        Task task2 = DoWorkAsync(2);
        Task task3 = DoWorkAsync(3);

        // Čekání na dokončení všech úloh
        await Task.WhenAll(task1, task2, task3);
    }

    private async Task DoWorkAsync(int taskNumber)
    {
        // Simulace nějaké práce
        for (int i = 0; i < 10; i++)
        {
            System.Console.WriteLine($"Úloha {taskNumber}: iterace {i}");
            await Task.Delay(1000); // Pauza 1 sekunda
        }
    }
}
```

- Spustit v synchronním módu

```
using System.Threading.Tasks;

public class TPLExample
{
    public void RunTasks()
    {
        // Vytvoření a spuštění úloh paralelně
        Task task1 = Task.Run(() => DoWork(1));
        Task task2 = Task.Run(() => DoWork(2));
        Task task3 = Task.Run(() => DoWork(3));

        // Čekání na dokončení všech úloh
        Task.WaitAll(task1, task2, task3);
    }

    private void DoWork(int taskNumber)
    {
        // Simulace nějaké práce
        for (int i = 0; i < 10; i++)
        {
            System.Console.WriteLine($"Úloha {taskNumber}: iterace {i}");
            System.Threading.Thread.Sleep(1000); // Pauza 1 sekunda
        }
    }
}
```


Zjištění kde se kód vykonává

`Thread.CurrentThread` k získání odkazu na aktuální vlákno.

Poté lze zkontrolovat jeho vlastnosti, jako je `IsThreadPoolThread` nebo `IsBackground`, které mohou poskytnout další informace.

Tipy

Vytvořit vlastní LINQ

Lze toho dosáhnout skrze rozšíření metod a delegátů.

Příklad:

```
public static class MyExtensions
{
    public static IEnumerable<T> MyWhere<T>(this IEnumerable<T> collection, Func<T, bool> predicate)
    {
        foreach (var item in collection)
        {
            if (predicate(item))
            {
                yield return item;
            }
        }
    }
}
```

V tomto příkladu `MyWhere` je rozšíření metody pro `IEnumerable<T>`, což znamená, že ji můžete použít na jakoukoli kolekci.

Metoda přijímá delegáta `Func<T, bool>`, který je použit k rozhodnutí, zda prvek splňuje danou podmínku.

Můžete pak použít tuto metodu podobně jako metodu where v LINQ:

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
var evenNumbers = numbers.MyWhere(x => x % 2 == 0); // filtruje sudá čísla v seznamu.
```

Synchronizace vláken pomocí lock

Zabrání vstupu jiného vlákna do bloku kódu.

> **NOTE**
> Zajistí tedy, že blok kódu uvnitř `lock` může být v daném okamžiku spuštěn pouze jedním vláknem.

Příklad:

```
object zamek = new object();

lock (zamek)
{
    // Kód, který má být prováděn pouze jedním vláknem
}
```

Když vlákno vstoupí do bloku `lock`, získá zámek na objektu `zamek`.

Jakmile vlákno opustí `lock`, `zamek` je automaticky uvolněn.

WARNING

Použití `lock(this)` je nebezpečné, protože jiné části kódu mohou také získat zámek na `this` a způsobit deadlock.

Techniky (../techniques.html)

+ **Balíčky**

- **C# (../net.html)**
  - Interface (../net/net_interface.html)
  - Datové typy (../net/net_dataTypes.html)
  - Atributy (../net/net_attributes.html)
  - Metody (../net/net_methods.html)
  - Testy Nunit (../net/net_nunit.html)
  - Entity Framework (../net/net_entityFramework.html)
  - WPF (../net/net_wpf.html)
  - + **Soubory (../net/net_files.html)**

JavaScript (../javascript.html)

Nunit = Testovací framework.

Kód

Multiple Asserts

Nechceme, aby se test ukončil, jakmile první `assert` selže uvnitř metody.

- Ve standardním případě, pokud první test selže, tak následující testy již nejsou spuštěny z této metody.

> **NOTE**
> Stane se pouze v případě, že máme více `assert` v jedné metodě.

Příklad použití

```
[Test]
public void MultipleAssertsDemo()
{
    var situationUnderTest = new SomeCalculator();
    var result = situationUnderTest.DoCalculation();

    Assert.Multiple(() =>
    {
        Assert.That(result.RealPart, Is.EqualTo(5.2));
        Assert.That(result.ImaginaryPart, Is.EqualTo(3.9));
    });

    // can also work with the classic assertion syntax
    Assert.Multiple(() =>
    {
        ClassicAssert.AreEqual(5.2, result.RealPart, "Real Part");
        ClassicAssert.AreEqual(3.9, result.ImaginaryPart, "Imaginary Part");
    });
}
```

Video prezentace

Trendy v unit testování a mockování (WUG Days 2018) (https://download.wug.cz/videos/wug/WUGBrno_WUG-Days-2018_Trendy-v-unit-testovani-a-mockovani/WUGBrno_WUG-Days-2018_Trendy-v-unit-testovani-a-mockovani_1080p.mp4)

Instalace

```
"C:\Program Files\dotnet\dotnet.exe" tool install --ignore-failed-sources --global d
otnet-ef
```

Filter by title

Techniky (../techniques.html)

+ **Balíčky**

- **C# (../net.html)**

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ **Soubory (../net/net_files.html)**

JavaScript (../javascript.html)

NOTE

Balíček bude uložen ve složce: `C:\Users\<TvéUživatelskéJméno>\.dotnet\tools`

Pro zálohu offline, zkopírujte obsah této složky na jiný počítač, kde nástroj `dotnet-ef` nebude dostupný online.

WARNING

Pokud složku umístíte na jinou cestu, ujistěte se, že ji přidáte do proměnných do `PATH`, aby byl nástroj dostupný z příkazového řádku.

1. Spusťte build pro zobrazení chyb

```
dotnet build
```

2. Vytvořte první migraci

IMPORTANT

Ujistěte se, že se nacházíte ve složce, kde se nachází váš `.csproj` soubor

```
dotnet ef migrations add InitialCreate
```

3. Aktualizujte databázi pomocí migrace

```
dotnet ef database update
```

Příkazy

Příkaz	Popis
`dotnet ef migrations add <Název>`	Vytvoří nový soubor pro migraci s názvem `<Název>`, který zachytí změny ve tvých modelech (entitách).
`dotnet ef migrations remove`	Smaže poslední migraci, kterou jsi přidal, ale nezmění databázi (pouze vrátí kód zpět).
`dotnet ef migrations list`	Zobrazí seznam všech migrací, které jsi vytvořil (ukazuje, jaké změny se postupně prováděly).
`dotnet ef database update`	Aplikuje všechny migrace (změny) na databázi, aby se databáze aktualizovala podle aktuálních modelů.
`dotnet ef database update <Název>`	Aplikuje migraci s názvem `<Název>` (pokud nechceš aplikovat všechny migrace).
`dotnet ef database drop`	Smaže celou databázi – dávej pozor, tímto příkazem přijdeš o všechna data.
`dotnet ef dbcontext list`	Ukáže všechny třídy DbContext ve tvém projektu (DbContext je hlavní třída pro práci s databází).
`dotnet ef dbcontext info`	Zobrazí informace o tvojí DbContext třídě (užitečné pro zjištění detailů o konfiguraci).
`dotnet ef dbcontext scaffold`	Vytvoří třídy (modely) podle existující databáze – tímto způsobem můžeš začít, pokud už máš databázi.
`dotnet ef migrations script`	Vygeneruje SQL skript, který obsahuje všechny změny v migracích – vhodné pro manuální nasazení.

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

WPF (Windows Presentation Foundation)

- Tvorba desktopových aplikací na platformě Windows
- Odděluje logiku aplikace (C#) od vzhledu (XAML)
- Umožňuje datové vazby a stylování
- Podpora vektorové grafiky, animací a multimédií

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Filter by title

Prvky

Button

Tlačítko je interaktivní prvek, na který může uživatel kliknout.

Vlastnosti, které lze stylovat

- Background: Barva pozadí tlačítka.
- Foreground: Barva textu.
- BorderBrush: Barva ohraničení.
- BorderThickness: Tloušťka ohraničení.
- FontSize: Velikost písma.
- Padding: Vnitřní odsazení (rozestup textu od okrajů).
- Margin: Vnější odsazení (rozestup tlačítka od ostatních prvků).
- CornerRadius: Zaoblení rohů tlačítka.

Události

- Click: Vyvolána, když uživatel klikne na tlačítko.

Příklad použití

```
<Button
    Content="Klikni na mě"
    Background="LightBlue"
    Foreground="White"
    BorderBrush="Blue"
    BorderThickness="2"
    FontSize="16"
    Padding="10"
    Margin="10"
    CornerRadius="5"/>
```

TextBox

TextBox je prvek pro vstup textu od uživatele.

Vlastnosti, které lze stylovat

- Background: Barva pozadí vstupního pole.

- Foreground: Barva textu.
- BorderBrush: Barva ohraničení.
- BorderThickness: Tloušťka ohraničení.
- FontSize: Velikost písma.
- Padding: Vnitřní odsazení.
- Margin: Vnější odsazení.
- Width: Šířka pole.
- Height: Výška pole.

Události

- TextChanged: Vyvolána, když se změní obsah `TextBox`.
- GotFocus: Vyvolána, když `TextBox` získá fokus.
- LostFocus: Vyvolána, když `TextBox` ztratí fokus.

Příklad použití

```
<TextBox
    Text="Zadejte text"
    Background="White"
    Foreground="Black"
    BorderBrush="Gray"
    BorderThickness="1"
    FontSize="14"
    Padding="5"
    Margin="10"
    Width="200"
    Height="30"
    TextChanged="TextBox_TextChanged"
    GotFocus="TextBox_GotFocus"
    LostFocus="TextBox_LostFocus"/>
```

CheckBox

CheckBox je prvek, který umožňuje uživateli vybrat nebo odznačit volbu.

Vlastnosti, které lze stylovat

- Background: Barva pozadí.
- Foreground: Barva textu.
- BorderBrush: Barva ohraničení.
- BorderThickness: Tloušťka ohraničení.
- FontSize: Velikost písma.
- Padding: Vnitřní odsazení.
- Margin: Vnější odsazení.

Události

- Checked: Vyvolána, když uživatel zaškrtne `CheckBox`.
- Unchecked: Vyvolána, když uživatel odškrtne `CheckBox`.
- Click: Vyvolána, když uživatel klikne na `CheckBox`.

Příklad použití

```
<CheckBox
    Content="Souhlasím s podmínkami"
    Background="Transparent"
    Foreground="Black"
    BorderBrush="Gray"
    BorderThickness="1"
    FontSize="14"
    Padding="5"
    Margin="10"
    Checked="CheckBox_Checked"
    Unchecked="CheckBox_Unchecked"
    Click="CheckBox_Click"/>
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

ComboBox

ComboBox je prvek, který umožňuje výběr jedné hodnoty z rozevíracího seznamu.

Vlastnosti, které lze stylovat

- Background: Barva pozadí.
- Foreground: Barva textu.
- BorderBrush: Barva ohraničení.
- BorderThickness: Tloušťka ohraničení.
- FontSize: Velikost písma.
- Padding: Vnitřní odsazení.
- Margin: Vnější odsazení.
- Width: Šířka pole.
- Height: Výška pole.

Události

- SelectionChanged:Vyvolána, když uživatel vybere novou položku v `ComboBox`.
- DropDownOpened: Vyvolána, když je rozevírací seznam otevřen.
- DropDownClosed: Vyvolána, když je rozevírací seznam zavřen.

Příklad použití

```
<ComboBox
    Background="White"
    Foreground="Black"
    BorderBrush="Gray"
    BorderThickness="1"
    FontSize="14"
    Padding="5"
    Margin="10"
    Width="150"
    Height="30"
    SelectionChanged="ComboBox_SelectionChanged"
    DropDownOpened="ComboBox_DropDownOpened"
    DropDownClosed="ComboBox_DropDownClosed">
    <ComboBoxItem Content="Možnost 1"/>
    <ComboBoxItem Content="Možnost 2"/>
    <ComboBoxItem Content="Možnost 3"/>
</ComboBox>
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

RadioButton

RadioButton je prvek, který umožňuje uživateli vybrat jednu možnost z více možností.

Vlastnosti, které lze stylovat

- Background: Barva pozadí.
- Foreground: Barva textu.
- BorderBrush: Barva ohraničení.
- BorderThickness: Tloušťka ohraničení.
- FontSize: Velikost písma.
- Padding: Vnitřní odsazení.
- Margin: Vnější odsazení.

Události

- Checked: Vyvolána, když je `RadioButton` vybrán.
- Unchecked: Vyvolána, když je `RadioButton` odznačen.
- Click: Vyvolána, když uživatel klikne na `RadioButton`.

Příklad použití

```
<StackPanel Margin="10">
    <TextBlock Text="Vyberte si jednu z možností:" FontSize="16" Margin="0,0,0,10"/>

    <RadioButton
        Content="Možnost A"
        GroupName="OptionsGroup"
        Background="LightGray"
        Foreground="Black"
        BorderBrush="DarkGray"
        BorderThickness="2"
        FontSize="14"
        Padding="8"
        Margin="0,5"
        CornerRadius="10"
        Checked="RadioButton_Checked"
        Unchecked="RadioButton_Unchecked"
        Click="RadioButton_Click"/>

    <RadioButton
        Content="Možnost B"
        GroupName="OptionsGroup"
        Background="LightGray"
        Foreground="Black"
        BorderBrush="DarkGray"
        BorderThickness="2"
        FontSize="14"
        Padding="8"
        Margin="0,5"
        CornerRadius="10"
        Checked="RadioButton_Checked"
        Unchecked="RadioButton_Unchecked"
        Click="RadioButton_Click"/>
</StackPanel>
```


Slider

Slider je prvek, který umožňuje uživateli vybrat hodnotu posunutím jezdce.

Vlastnosti, které lze stylovat

- Background: Barva pozadí.
- Foreground: Barva jezdce.
- BorderBrush: Barva ohraničení.
- BorderThickness: Tloušťka ohraničení.
- Width: Šířka slideru.
- Height: Výška slideru.

Události

- ValueChanged: Vyvolána, když se změní hodnota `Slider`.
- GotFocus: Vyvolána, když `Slider` získá fokus.
- LostFocus: Vyvolána, když `Slider` ztratí fokus.

Příklad použití

```
<Slider
    Minimum="0"
    Maximum="100"
    Value="50"
    Background="LightGray"
    Foreground="Blue"
    Width="200"
    Height="30"
    Margin="10"
    ValueChanged="Slider_ValueChanged"
    GotFocus="Slider_GotFocus"
    LostFocus="Slider_LostFocus"/>
```

Vlastní ovládací prvek

Pokud standardní ovládací prvky nevyhovují vašim potřebám, můžete vytvořit vlastní ovládací prvek.

- **Definice vlastního ovládacího prvku**

  Vytvořte třídu vlastního ovládacího prvku. (Tato třída by měla dědit z existujícího ovládacího prvku, např. `Button`.)

```
using System.Windows;
using System.Windows.Controls;

namespace YourNamespace
{
    public class MyCustomButton : Button
    {
        static MyCustomButton()
        {
            DefaultStyleKeyProperty.OverrideMetadata(typeof(MyCustomButton),
                new FrameworkPropertyMetadata(typeof(MyCustomButton)));
        }
    }
}
```

  - **DefaultStyleKeyProperty**: Určuje výchozí styl pro vlastní ovládací prvek.
- **Definujte styl a šablonu pro vlastní ovládací prvek**

  Styl a šablonu můžete definovat v XAML jako obvykle.

```
<Window.Resources>
  <Style TargetType="{x:Type local:MyCustomButton}">
      <Setter Property="Background" Value="LightGray"/>
      <Setter Property="Foreground" Value="Black"/>
      <Setter Property="FontSize" Value="16"/>
      <Setter Property="Template">
          <Setter.Value>
              <ControlTemplate TargetType="{x:Type local:MyCustomButton}">
                  <Border Background="{TemplateBinding Background}"
                          BorderBrush="Black"
                          BorderThickness="2"
                          CornerRadius="10">
                      <ContentPresenter HorizontalAlignment="Center"
                                        VerticalAlignment="Center"/>
                  </Border>
              </ControlTemplate>
          </Setter.Value>
      </Setter>
  </Window.Resources>
```

- **Použití vlastního ovládacího prvku** v XAML:

```
<local:MyCustomButton Content="Moje vlastní tlačítko" Width="200" Height="50"/>
```

Styly

Styl se používá k definování vzhledu a chování více prvků najednou.

Definuje se pomocí **XAML**.

- Definování stylu

  Styl se definuje v sekci **Resources**:

```
<Window.Resources>
    <Style x:Key="MyButtonStyle" TargetType="Button">
        <Setter Property="Background" Value="Blue"/>
        <Setter Property="Foreground" Value="White"/>
        <Setter Property="FontSize" Value="14"/>
        <Setter Property="Padding" Value="10"/>
    </Style>
</Window.Resources>
```

  - **x**: Unikátní název stylu, který použijete později.
  - **TargetType**: Typ prvku, na který se styl vztahuje.
- Použití stylu

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

+ JavaScript (../javascript.html)

Chcete-li styl použít na prvek, využijete atribut **Style**:

```
<Button Style="{StaticResource MyButtonStyle}" Content="Klikni na mě"/>
```

Prefixy

x: Techniky (../techniques.html)

- Vyhrazen pro XAML standardní funkce a typy.
- Používá se pro přístup k základním vlastnostem, jako jsou `x:Class`, `x:Name`, `x:Key`, atd.
- Příklad:

```
<Window x:Class="MyNamespace.MainWindow"
        x:Name="mainWindow"
        x:Key="myWindowKey">
```

xmlns:

- Používá se k deklaraci namespace.
- Obvykle se používá v kořenovém prvku XAML souboru.
- Příklad deklarace namespace:

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:MyNamespace">
```

local: JavaScript (../javascript.html)

- Používán k odkazování na vlastní namespace aplikace.
- Můžete ho použít k přístupu k vlastním ovládacím prvkům, datovým modelům a dalším třídám definovaným ve vaší aplikaci.
- Příklad:

```
<local:MyCustomButton Content="Moje vlastní tlačítko"/>
```

xmlns:sys:

- Pro přístup k základním typům .NET, jako jsou `System.String`, `System.Int32`, atd.
- Příklad:

```
xmlns:sys="clr-namespace:System;assembly=mscorlib"
```

xmlns:controls:

- Pro přístup k ovládacím prvkům z externích knihoven, jako je například **Windows Community Toolkit**.
- Příklad:

+ Balíčky
- C# (../net.html)
  - Interface (../net/net_interface.html)
  - Datové typy (../net/net_dataTypes.html)
  - Atributy (../net/net_attributes.html)
  - Metody (../net/net_methods.html)
  - Testy Nunit (../net/net_nunit.html)
  - Entity Framework (../net/net_entityFramework.html)
  - WPF (../net/net_wpf.html)
+ Soubory (../net/net_files.html)

```
        <controls:MyCustomControl/>
```

xmlns:mc:

- Používá se pro **Markup Compatibility**.
- Umožňuje použití starších XAML formátů a zajišťuje zpětnou kompatibilitu.
- Příklad:

```
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
```

xmlns:d:

- Používá se pro návrhové časové funkce a umožňuje definovat prvky, které se zobrazují pouze během návrhu.
- Příklad:

```
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
```

Použití prefixů v XAML

```
<Window x:Class="MyNamespace.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:MyNamespace"
        xmlns:controls="clr-namespace:MyCustomControls;assembly=MyCustomControlsAsse
mbly"
        xmlns:sys="clr-namespace:System;assembly=mscorlib"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        mc:Ignorable="d"
        Title="Hlavní okno" Height="350" Width="525">

    <Grid>
        <local:MyCustomButton Content="Moje vlastní tlačítko" Width="200" Height="5
0"/>
        <Button Content="Tlačítko" Width="100" Height="30"/>
    </Grid>
</Window>
```

Šablony (ControlTemplates)

Šablony umožňují plně přizpůsobit vzhled ovládacího prvku.

Šablona definuje strukturu a vzhled prvku.

- Vytvoření šablony pro tlačítko

```
<Window.Resources>
  <ControlTemplate x:Key="MyButtonTemplate" TargetType="Button">
      <Border Background="{TemplateBinding Background}"
              BorderBrush="Black"
              BorderThickness="2"
              CornerRadius="5">
          <ContentPresenter HorizontalAlignment="Center"
                            VerticalAlignment="Center"/>
      </Border>
  </ControlTemplate>
</Window.Resources>
```

- **ControlTemplate**: Určuje, jak bude tlačítko vypadat.
- **TemplateBinding**: Slouží k vázání vlastností stylu na vlastnosti šablony.

```
<Button Template="{StaticResource MyButtonTemplate}"
        Background="LightBlue"
        Content="Stylizované tlačítko"/>
```

Responzivní design prvků

Responzivní design znamená, že se aplikace přizpůsobí různým velikostem a rozlišením obrazovky.

Layout Panely

WPF nabízí různé layout panely, které vám pomohou uspořádat ovládací prvky, jako jsou tlačítka nebo textová pole.

Zde jsou nejčastěji používané panely:

- **Grid**:

  Rozděluje okno na řádky a sloupce.

  Umožňuje flexibilní uspořádání.

  Příklad:

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="*" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <Button Grid.Row="0" Grid.Column="0" Content="Tlačítko 1" />
    <Button Grid.Row="0" Grid.Column="1" Content="Tlačítko 2" />
    <Button Grid.Row="1" Grid.Column="0" Content="Tlačítko 3" />
    <Button Grid.Row="1" Grid.Column="1" Content="Tlačítko 4" />
</Grid>
```

V tomto příkladu je **Grid** rozdělen na 2 řádky a 2 sloupce, každá buňka obsahuje jedno tlačítko.

- **StackPanel**

  Ukládá ovládací prvky buď vertikálně (jeden pod druhým), nebo horizontálně (vedle sebe).

  Příklad:

```
<StackPanel Orientation="Vertical">
    <Button Content="Tlačítko 1" />
    <Button Content="Tlačítko 2" />
    <Button Content="Tlačítko 3" />
</StackPanel>
```

Zde jsou tlačítka naskládána vertikálně (jedno pod druhým).

Pokud chceme prvky uspořádat horizontálně:

```
<StackPanel Orientation="Horizontal">
    <Button Content="Tlačítko 1" />
    <Button Content="Tlačítko 2" />
    <Button Content="Tlačítko 3" />
</StackPanel>
```

- **WrapPanel**:

  Pokud je na obrazovce málo místa, ovládací prvky se "obalí" na další řádek/sloupec.

  Příklad:

```
<WrapPanel>
    <Button Content="Tlačítko 1" Width="100" Height="50" />
    <Button Content="Tlačítko 2" Width="100" Height="50" />
    <Button Content="Tlačítko 3" Width="100" Height="50" />
    <Button Content="Tlačítko 4" Width="100" Height="50" />
    <Button Content="Tlačítko 5" Width="100" Height="50" />
</WrapPanel>
```

- **DockPanel**:

  Umožňuje umístit ovládací prvky na okraje okna (vlevo, vpravo, nahoře, dole) a zbývající prostor zaplní jedno prvkem.

  Příklad:

```
<DockPanel>
    <Button DockPanel.Dock="Top" Content="Horní tlačítko" />
    <Button DockPanel.Dock="Bottom" Content="Spodní tlačítko" />
    <Button DockPanel.Dock="Left" Content="Levé tlačítko" />
    <Button DockPanel.Dock="Right" Content="Pravé tlačítko" />
    <Button Content="Centrální tlačítko" />
</DockPanel>
```

Dynamické Velikosti

Dynamické velikosti znamenají, že se ovládací prvky automaticky přizpůsobují velikosti okna.

Místo pevně definovaných hodnot můžete použít:

- **Procentuální hodnoty**

  Umožňují ovládacím prvkům zabírat procento rodičovského prvku.

  Příklad:

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="50%"/> <!-- 50% šířky rodičovského prvku -->
        <ColumnDefinition Width="50%"/> <!-- 50% šířky rodičovského prvku -->
    </Grid.ColumnDefinitions>

    <TextBlock Text="Toto zabírá 50% šířky okna" Background="LightCoral" Grid.C
olumn="0" />
    <TextBlock Text="Toto také zabírá 50% šířky okna" Background="LightBlue" Gr
id.Column="1" />
</Grid>
```

- **Hodnoty s `*`**

  V Gridu můžete použít `*` k rozdělení zbývajícího místa.

Například `2*` a `1*` znamená, že první sloupec zabere dvakrát více místa než druhý.

Příklad:

```
<Grid>
  <Grid.ColumnDefinitions>
      <ColumnDefinition Width="2*"/> <!-- Dva díly prostoru -->
      <ColumnDefinition Width="1*"/> <!-- Jeden díl prostoru -->
  </Grid.ColumnDefinitions>

  <TextBlock Text="Toto zabírá 2/3 šířky okna" Background="LightCoral" Grid.Column="0"/>
  <TextBlock Text="Toto zabírá 1/3 šířky okna" Background="LightBlue" Grid.Column="1"/>
</Grid>
```

- **Auto**
  Automatická velikost na základě obsahu.
  Příklad:

```
<Grid>
  <Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>  <!-- Automatická výška pro hlavičku -->
      <RowDefinition Height="*"/>     <!-- Zbytek prostoru pro další prvky -->
  </Grid.RowDefinitions>

  <TextBlock Text="Hlavička" Grid.Row="0" FontSize="24" Background="LightGray" />
  <Button Content="Toto tlačítko zabírá zbytek prostoru" Grid.Row="1" />
</Grid>
```

Sledování Změny Velikosti

Můžete sledovat změny velikosti okna a upravit rozložení ovládacích prvků.

Pomocí události `SizeChanged` můžete reagovat na změnu velikosti okna a provést potřebné úpravy.

Příklad:

```
private void Window_SizeChanged(object sender, SizeChangedEventArgs e)
{
    // Získání nové šířky a výšky okna
    double newWidth = e.NewSize.Width;
    double newHeight = e.NewSize.Height;

    // Například upravit velikost tlačítka na základě velikosti okna
    if (newWidth < 600)
    {
        myButton.Width = 100; // Menší šířka tlačítka
    }
    else
    {
        myButton.Width = 200; // Větší šířka tlačítka
    }
    // Můžete také změnit další vlastnosti na základě velikosti okna
}
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

IMPORTANT

Nezapomeňte přidat událost do XAML:

```
<Window x:Class="ResponzivniDesign.MainWindow"
        SizeChanged="Window_SizeChanged">
```

ViewBox

Vše, co je uvnitř ViewBoxu, se přizpůsobí velikosti okna. (Automatické škálování obsahu.)

Příklad:

```
<ViewBox>
    <Grid>
        <TextBlock Text="Toto je responzivní text!" FontSize="20"/>
    </Grid>
</ViewBox>
```

Triggery

Triggery umožňují dynamicky měnit vzhled prvku na základě určitých událostí nebo podmínek.

- Použití triggeru

  Zde je příklad stylu tlačítka, který mění barvu pozadí, když je kurzor myši nad tlačítkem:

```
        <Style TargetType="Button">
            <Setter Property="Background" Value="Gray"/>
            <Setter Property="Foreground" Value="White"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="Green"/>
                </Trigger>
            </Style.Triggers>
        </Style>
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Data Binding (Vazba Modelu na View)

1. Vytvoření ViewModel

Nejprve vytvoříme ViewModel, který bude obsahovat vlastnost, kterou chceme vázat.

IMPORTANT

Použijeme `INotifyPropertyChanged` , aby WPF věděl, kdy se vlastnost změnila.

```
using System.ComponentModel;

public class MyViewModel : INotifyPropertyChanged
{
    private string _name;

    public string Name
    {
        get { return _name; }
        set
        {
            _name = value;
            OnPropertyChanged(nameof(Name));
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

NOTE

`MyViewModel` má vlastnost `Name` , která implementuje `INotifyPropertyChanged` .

To zajišťuje, že pokud se `Name` změní, UI se automaticky aktualizuje.

2. Vytvoření XAML pro UI

Vytvoříme jednoduché uživatelské rozhraní, které umožní uživateli zadat jméno a zobrazit ho.

Použijeme `TextBox` pro zadání a `TextBlock` pro zobrazení.

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Binding Example" Height="200" Width="300">
    <Grid>
        <TextBox Text="{Binding Name, UpdateSourceTrigger=PropertyChanged}" Width="200" Margin="10"/>
        <TextBlock Text="{Binding Name}" Margin="10,50,10,10"/>
    </Grid>
</Window>
```

> **NOTE**
>
> V XAML používáme `{Binding Name}` pro vázání `TextBox` a `TextBlock` na vlastnost `Name` ve ViewModelu.
>
> `UpdateSourceTrigger=PropertyChanged` znamená, že binding se aktualizuje při každé změně textu v `TextBox`.
>
> (Vlastnost ViewModelu se aktualizuje okamžitě při každé změně textu.)
>
> Příklad:
>
> ```
> <TextBox Text="{Binding Name, UpdateSourceTrigger=PropertyChanged}" Width="200"
> Margin="10"/>
> ```
>
> Bez explicitního nastavení `UpdateSourceTrigger` se binding aktualizuje při ztrátě fokusu (`LostFocus`).
>
> (Vlastnost ViewModelu se aktualizuje pouze po ztrátě fokusu či při změně výběru v ComboBox.)
>
> Příklad:
>
> ```
> <TextBox Text="{Binding Name}" Width="200" Margin="10"/>
> ```

3. Nastavení DataContext

V `MainWindow.xaml.cs` nastavíme `DataContext` na instanci našeho `ViewModelu` .

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

```
using System.Windows;

namespace WpfApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            DataContext = new MyViewModel(); // Nastavujeme DataContext
        }
    }
}
```

NOTE

V konstruktoru `MainWindow` nastavujeme `DataContext` na instanci `MyViewModel`.
To umožňuje XAML binding k vlastnostem ViewModelu.

Validace

Implementace

- `INotifyPropertyChanged` + `IDataErrorInfo`
  - Obvykle se používá pro vracení celkové chyby objektu.

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

```
public class MyViewModel : INotifyPropertyChanged, IDataErrorInfo
{
    private string _name;

    // Property s validací
    [Required(ErrorMessage = "Pole je povinné.")]
    public string Name
    {
        get => _name;
        set
        {
            _name = value;
            OnPropertyChanged(nameof(Name)); // Oznámení o změně
        }
    }

    // Notifikace o změně
    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyNam
e));
    }

    // Implementace IDataErrorInfo
    public string this[string columnName]
    {
        get
        {
            var validationResults = new List<ValidationResult>();
            var context = new ValidationContext(this) { MemberName = columnName
};
            Validator.TryValidateProperty(
                this.GetType().GetProperty(columnName).GetValue(this),
                context,
                validationResults
            );

            // Vrátí první chybu, pokud existuje, jinak vrátí null
            return validationResults.FirstOrDefault()?.ErrorMessage;
        }
    }

    public string Error
    {
        get
        {
            return null; // Gets a message that describes any validation errors
for the object.
        }
```

```
        }
    }
```

> **NOTE**
>
> Použití `this[string columnName]`:
>
> Tato metoda slouží k validaci konkrétní vlastnosti.
>
> Pokud je chyba nalezena, vrátí odpovídající chybovou zprávu.

Technika (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

- **`INotifyPropertyChanged + INotifyDataErrorInfo`**

  Umožňuje validovat více chyb na jedné vlastnosti a spravovat chybové zprávy pro každou vlastnost asynchronně.

XAML pro validaci pomocí `IDataErrorInfo`

```
<TextBox Text="{Binding Name,
                UpdateSourceTrigger=PropertyChanged,
                ValidatesOnDataErrors=True}" />

<TextBox.Style>
    <Style TargetType="TextBox">
        <Style.Triggers>
            <Trigger Property="Validation.HasError" Value="True">
                <Setter Property="BorderBrush" Value="Red" />
                <Setter Property="BorderThickness" Value="2" />
            </Trigger>
        </Style.Triggers>
    </Style>
</TextBox.Style>
```

```
public class MyViewModel : INotifyPropertyChanged, INotifyDataErrorInfo
{
    private string _name;
    private readonly Dictionary<string, List<string>> _errors = new Dictionary<
string, List<string>>();

    // Property s validací
    [Required(ErrorMessage = "Pole je povinné.")]
    [StringLength(10, ErrorMessage = "Maximálně 10 znaků.")]
    public string Name
    {
        get => _name;
        set
        {
            _name = value;
            OnPropertyChanged(nameof(Name)); // Oznámení o změně
            ValidateProperty(nameof(Name), value); // Spuštění validace
        }
    }

    // Notifikace o změně
    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyNam
e));
    }

    // Validace vlastnosti
    private void ValidateProperty(string propertyName, object value)
    {
        var context = new ValidationContext(this) { MemberName = propertyName
};
        var validationResults = new List<ValidationResult>();

        bool isValid = Validator.TryValidateProperty(value, context, validation
Results);

        if (!isValid)
        {
            _errors[propertyName] = validationResults.Select(vr => vr.ErrorMess
age).ToList();
        }
        else
        {
            _errors.Remove(propertyName);
        }

        OnErrorsChanged(propertyName);
    }
```

```
        // Implementace INotifyDataErrorInfo
        public event EventHandler<DataErrorsChangedEventArgs> ErrorsChanged;

        // Vrací, zda má objekt nějaké chyby
        public bool HasErrors => _errors.Any();

        // Získá seznam chyb pro konkrétní vlastnost
        public IEnumerable GetErrors(string propertyName)
        {
            if (_errors.ContainsKey(propertyName))
                return _errors[propertyName];
            return null;
        }

        protected void OnErrorsChanged(string propertyName)
        {
            ErrorsChanged?.Invoke(this, new DataErrorsChangedEventArgs(propertyNam
        }
    }
}
```

XAML pro validaci pomocí `INotifyDataErrorInfo`

```
<TextBox Text="{Binding Name,
                UpdateSourceTrigger=PropertyChanged,
                ValidatesOnNotifyDataErrors=True}" />

<TextBox.Style>
    <Style TargetType="TextBox">
        <Style.Triggers>
            <Trigger Property="Validation.HasError" Value="True">
                <Setter Property="BorderBrush" Value="Red" />
                <Setter Property="BorderThickness" Value="2" />
            </Trigger>
        </Style.Triggers>
    </Style>
</TextBox.Style>
```

Animace

WPF podporuje animace, které umožňují měnit vlastnosti prvků v čase.

Zde je příklad, jak animovat změnu barvy pozadí tlačítka, když na něj najedete:

```
<Button Content="Klikni na mě">
    <Button.Triggers>
        <EventTrigger RoutedEvent="Button.MouseEnter">
            <BeginStoryboard>
                <Storyboard>
                    <ColorAnimation Storyboard.TargetProperty="(Button.Background).
(SolidColorBrush.Color)"
                                    To="Red" Duration="0:0:1"/>
                </Storyboard>
            </BeginStoryboard>
        </EventTrigger>
    </Button.Triggers>
</Button>
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)

Datové typy (../net/net_dataTypes.html)

Atributy (../net/net_attributes.html)

Metody (../net/net_methods.html)

Testy Nunit (../net/net_nunit.html)

Entity Framework (../net/net_entityFramework.html)

WPF (../net/net_wpf.html)

+ Soubory (../net/net_files.html)

JavaScript (../javascript.html)

Filter by title

Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

- Soubory (../net/net_files.html)

XML (../net/net_xml.html)

JavaScript (../javascript.html)

Vyhledat oddělovač

```
public static StreamReader FindDelimiter(StreamReader reader, out char delimiter, int? linesToRead = null)
{
    const char semicolon = ';';
    const char comma = ',';
    Dictionary<char, int> delimiters = new Dictionary<char, int>
    {
        { semicolon, default(int) },
        { comma, default(int) },
    };
    string line;
    int linesRead = 0;
    while ((line = reader.ReadLine()) != null && (!linesToRead.HasValue || linesRead < linesToRead.Value))
    {
        foreach (char c in line)
        {
            switch (c)
            {
                case semicolon:
                    delimiters[semicolon]++;
                    break;
                case comma:
                    delimiters[comma]++;
                    break;
            }
        }

        linesRead++;
    }

    delimiters = delimiters.Where(i => i.Value != 0).ToDictionary(i => i.Key, i => i.Value);

    if (delimiters.Count == 0)
        throw new Exception("Nepodařilo se dohledat jakýkoli oddělovač.");

    var highest = delimiters.Aggregate((item1, item2) => item1.Value > item2.Value ? item1 : item2);

        // Kontrola, zda existuje jiný oddělovač, který se vyskytuje alespoň v 70% případů jako nejčastější oddělovač
    const int failPercentage = 70;
    if ((from val in delimiters
         where val.Key != highest.Key
         select new decimal(val.Value) / new decimal(highest.Value) * 100).Any(diff
```

```
=> diff >= failPercentage))
        throw new Exception("Typ oddělovače se nepodařilo jednoznačne identifikova
t.");

    delimiter = highest.Key;

    reader.DiscardBufferedData();
    reader.BaseStream.Seek(0, System.IO.SeekOrigin.Begin);
    return reader;
}
```


Escape sekvence

- `\uFEFF`
  = **Byte Order Mark (BOM)**.
  BOM nám říká, jak číst data v souboru - od začátku nebo od konce.

  WARNING
  Může způsobit problémy s některými nástroji a knihovnami, které jej neočekávají.

- `\u0000`
  = **Null znak**
  Null znak je speciální znak, který se často používá k označení konce řetězce v některých programovacích jazycích a systémech.

  WARNING
  Je obvykle nepotřebný a může způsobit problémy při parsování nebo zpracování dat.

Náhrada znaků

Základní znaky:

- < (levá ostrá závorka

  `<`

- > (pravá ostrá závorka)

  `>`

- & (ampersand)

  `&`

- ' (apostrof)

  `'`

- " (uvozovky)

  `"`

Znaky s diakritikou:

- Á

  `Á`

- á

  `á`

- Č

  `Č`

- č

  `č`

> **NOTE**
>
> Stejným způsobem lze následně nahradit i další znaky s diakritikou.
>
> s prefixem `caron` (uvozovky) nebo `acute` (čárka nad písmenem).

Serializace a Deserializace objektu

Serializace

Proces, kdy se objekt převede na XML soubor.

```
public string SerializeObject(MyObject myObject)
{
    var serializer = new XmlSerializer(typeof(MyObject));
    using (var stringWriter = new StringWriter())
    {
        serializer.Serialize(stringWriter, myObject);
        return stringWriter.ToString();
    }
}
```


Deserializace

Proces, kdy se XML soubor převede zpět na objekt.

```
public MyObject DeserializeObject(string xml)
{
    var serializer = new XmlSerializer(typeof(MyObject));
    using (var stringReader = new StringReader(xml))
    {
        return (MyObject)serializer.Deserialize(stringReader);
    }
}
```

Namespace

Způsob jak zabránit konfliktu jmen v XML souborech.

NOTE

Poznáme podle klíčového slova `xmlns` u atributu.

WARNING

Pokud je namespace použit, musí se uvést i při zpracování souboru.

NOTE

URL namespace v XML dokumentu nemusí být skutečná existující URL adresa.

Je to pouze jedinečný identifikátor, který se používá k rozlišení názvů elementů a atributů v XML dokumentu.

Tento identifikátor je často ve formě URL pro snadnou identifikaci a jedinečnost, ale nemusí to být nutně platná URL adresa, kterou byste mohli otevřít v prohlížeči.

Například:

```
<?xml version="1.0" encoding="utf-16"?>
<p:Person xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="htt
p://www.w3.org/2001/XMLSchema" xmlns:p="test">
    <p:FirstName>John</p:FirstName>
    <p:LastName>Doe</p:LastName>
    <p:Age>30</p:Age>
</p:Person>
```



```
XmlSerializer s = new XmlSerializer(person.GetType());
StringBuilder sb = new StringBuilder();
using (StringWriter writer = new StringWriter(sb))
{
    XmlSerializerNamespaces ns = new XmlSerializerNamespaces();
    ns.Add("p", "test");
    s.Serialize(writer, person, ns);
}
```

XML soubor bez namespace

```
<person>
    <name>John Doe</name>
    <age>30</age>
</person>
```

WARNING

Vlastnosti na objektu **nesmí mít atribut `XmlElement` s namespace**.

```
XmlSerializer s = new XmlSerializer(person.GetType());
XmlSerializerNamespaces ns = new XmlSerializerNamespaces();
ns.Add("", "");
StringBuilder sb = new StringBuilder();
using (StringWriter writer = new StringWriter(sb))
{
    s.Serialize(writer, person, ns);
}
```

NOTE

Řešením jak odebrat namespace úplně všude a nebere v potaz atributy `XmlElement` s namespace.

```
XDocument document = XDocument.Parse(dataOutput);
if (document.Root == null) return;
foreach (var element in document.Root.DescendantsAndSelf())
{
    element.Name = element.Name.LocalName;
    element.ReplaceAttributes(element.Attributes()
            .Where(x => !x.IsNamespaceDeclaration)
            .Select(x => new XAttribute(x.Name.LocalName, x.Value)));
}
dataOutput = document.ToString();
```

Pro více info zde (https://stackoverflow.com/questions/10438284/how-to-strip-namespaces-from-xml-document).

XML soubor s namespace

Příklad 1:

```
<?xml version="1.0" encoding="utf-16"?>
<Person xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <FirstName>John</FirstName>
    <LastName>Doe</LastName>
    <Age>30</Age>
</Person>
```

```
XmlSerializer s = new XmlSerializer(person.GetType());
StringBuilder sb = new StringBuilder();
using (StringWriter writer = new StringWriter(sb))
{
    s.Serialize(writer, person);
}
```

Příklad 2:

```
<?xml version="1.0" encoding="utf-16"?>
<p:Person xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:p="http://www.example.com">
    <p:FirstName>John</p:FirstName>
    <p:LastName>Doe</p:LastName>
    <p:Age>30</p:Age>
</p:Person>
```

```
XmlSerializer s = new XmlSerializer(person.GetType());
StringBuilder sb = new StringBuilder();
using (StringWriter writer = new StringWriter(sb))
{
    XmlSerializerNamespaces ns = new XmlSerializerNamespaces();
    ns.Add("p", "http://www.example.com");
    s.Serialize(writer, person, ns);
}
```

NOTE

V tomto kódu "p" je prefix pro namespace a "http://www.example.com" je URL namespace.

Konvence serializace XML

Umožňuje přizpůsobit jak se třídy a vlastnosti serializují do XML.

Kořenový element

Označí třídu, která se má serializovat jako kořenový element

Pojmenuje kořenový element.

```
[XmlRoot("MyClass")]
public class MyClass
{
    ...
}
```

Ignorovat vlastnosti

Označuje vlastnosti, která se nemají serializovat.

```
[XmlIgnore]
public string MyProperty { get; set; }
```

Povolit jmenné prostory

Označuje, že se mají jmenné prostory serializovat jako atributy.

```
[XmlNamespaceDeclarations]
public class MyClass
{
    // ...
}
```

- **Techniky (../techniques.html)**
- **Balíčky**
- **C# (../net.html)**
  - Interface (../net/net_interface.html)
  - Datové typy (../net/net_dataTypes.html)
  - Atributy (../net/net_attributes.html)
  - Metody (../net/net_methods.html)
  - Testy Nunit (../net/net_nunit.html)
  - Entity Framework (../net/net_entityFramework.html)
  - WPF (../net/net_wpf.html)
  - **Soubory (../net/net_files.html)**
  - XML (../net/net_xml.html)
- **JavaScript (../javascript.html)**

Povolit/Zakázat přes metodu

ShouldSerialize{PropertyName} je metoda, která se volá při serializaci objektu.

Tato metoda se používá k rozhodnutí, zda se daná vlastnost má serializovat.

- Pokud metoda vrátí `true` vlastnost `{PropertyName}` se serializuje
- Pokud metoda vrátí `false` vlastnost `{PropertyName}` se neserializuje

```
public bool ShouldSerializeMyProperty()
{
    // logika rozhodnutí zda serializovat MyProperty
}
```

Povolit/Zakázat přes vlastnost bool

`{PropertyName}Specified` je vlastnost typu `bool` , která se používá k rozhodnutí, zda se má vlastnost serializovat.

Toto je alternativa k `ShouldSerialize{PropertyName}` .

- `{PropertyName}Specified` pokud je `true`

  vlastnost `{PropertyName}` se serializuje

- `{PropertyName}Specified` pokud je `false`

  vlastnost `{PropertyName}` se neserializuje

```
public bool MyPropertySpecified { get; set; }
```

Pojmenování kolekce a položek v kolekci

Používá se k pojmenování kolekce a položek v kolekci.

```
[XmlArray("MyCollection"), XmlArrayItem("Item")]
public List<string> MyProperty { get; set; }
```

Xml element ➡ xml atribut

Vlastnost, ktera se serializují jako "XML atribut" místo jako "XML element".

```
[XmlAttribute]
public string MyProperty { get; set; }
```

Xml element ➡ textový obsah

Označuje vlastnost, která má serializovat jako obsah XML elementu.

> To znamená, že vlastnost se serializuje jako textový obsah XML elementu, nikoli jako samostatný element.

```
[XmlText]
public string MyProperty { get; set; }
```

Enum ➡ xml element

Označuje výčtový typ, který se má serializovat jako XML element.

```
public enum MyEnum
{
    [XmlEnum("Value1")]
    Value1,
    [XmlEnum("Value2")]
    Value2
}
```

Třída ➡ xml element

Označí třídu, která se má serializovat jako XML element.

```
[XmlType("MyClass")]
public class MyClass
{
    // ...
}
```

Třída ze které se dědí ➡ xml element

Označuje třídy, které se mají serializovat jako potomky rodičovské třídy.

> To znamená, že pokud máte třídu `MyBaseClass` a od ní odvozenou třídu `MyDerivedClass`, musíte označit `MyBaseClass` pomocí `XmlIncludeAttribute`.

```
[XmlInclude(typeof(MyDerivedClass))]
public class MyBaseClass
{
    // ...
}

public class MyDerivedClass : MyBaseClass
{
    // ...
}
```

Libovolný xml element

Označuje vlastnost, která může obsahovat libovolný XML element.

> 👉 **WARNING**
>
> Musí být typu `XmlElement[]`, nebo `XmlElement`

```
[XmlAnyElement]
public XmlElement[] MyProperty { get; set; }
```

Libovolný xml atribut

Označuje vlastnost, která může obsahovat libovolný XML atribut.

> 👉 **WARNING**
>
> Musí být typu `XmlAttribute[]`, nebo `XmlAttribute`

```
[XmlAnyAttribute]
public XmlAttribute[] MyProperty { get; set; }
```


Techniky (../techniques.html)

+ Balíčky

- C# (../net.html)

Interface (../net/net_interface.html)
Datové typy (../net/net_dataTypes.html)
Atributy (../net/net_attributes.html)
Metody (../net/net_methods.html)
Testy Nunit (../net/net_nunit.html)
Entity Framework (../net/net_entityFramework.html)
WPF (../net/net_wpf.html)

- Soubory (../net/net_files.html)

XML (../net/net_xml.html)

JavaScript (../javascript.html)

Balíčky

Umístění

- Globální balíčky:
  - Windows:

    ```
    C:\Users\<user>\AppData\Roaming\npm\node_modules
    ```

Záloha

1. Vytvořit soubor s informacemi o nainstalovaných balíčcích.
   - Záloha globálních balíčků

     ```bash
     npm list -g --depth=0 > npm_global_packages.txt
     ```

     - Záloha lokálních balíčků

       ```
       npm list --depth=0 > npm_local_packages.txt
       ```

2. Vytvořit .tgz soubory pro balíčky.
   - Automaticky

```
# Read the file content
$content = Get-Content -Path 'npm_global_packages.txt'

# Skip the first line
$content = $content | Select-Object -Skip 1

# For each line in the content
foreach ($line in $content)
{
    # Remove unwanted symbols
    $line = $line -replace '[+`-]', ''

    # Split the line by '@' to separate the package name and version
    $parts = $line -split '@'

    # The first part is the package name
    $packageName = $parts[0].Trim()

    # Ignore lines that don't contain a package name or are the names of y
our text files
    if ($packageName -ne "" -and $packageName -ne "globalpackages.txt" -an
d $packageName -ne "npm_global_packages.txt")
    {
        # Run npm pack for the package
        npm pack $packageName
    }
}
```

> `'path-to-your-file.txt'`
>
> Cesta k souboru s informacemi o balíčcích.

- Manuálně

Pro každý balíček vytvoříme zálohu.

```
npm pack <package_name>
```

> `<package_name>`
>
> Název balíčku, který chceme zazálohovat.
>
> Například: `npm pack sass@1.76.0`

Obnova

> ⚠ **IMPORTANT**
> Při instalaci balíčku z `.tgz` je tento soubor následně automaticky odstraněn.

1. Instalace balíčků z .tgz souborů.
   - Automaticky

```
# Get all .tgz files in the current directory
$files = Get-ChildItem -Path .\ -Filter *.tgz

# Loop through each file
foreach ($file in $files)
{
    # Run npm install for the file
    Invoke-Expression -Command "npm install $($file.FullName)"
}
```

> Spustit ve složce s .tgz soubory.

   - Manuálně

```
npm install <package_path>
```

> Ceta k balíčku, který chceme obnovit.
>
> Například: `npm install sass-1.76.0.tgz`

Techniky (techniques.html)

+ Balíčky

+ C# (net.html)

JavaScript (javascript.html)